TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 11: PROGRAMACIÓN DEL LADO DEL CLIENTE: ADO Y ADO.NET CON VISUAL BASIC, C#, C++

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 06-11-26

1. Introducción y objetivo

Este informe explica qué son ADO y ADO.NET, cómo funcionan, en qué se diferencian y cómo se usan desde Visual Basic, C# y C++. Después describe el desarrollo de un módulo cliente para el proyecto IoT Analytics Dashboard de NexoDev Solutions, que realiza operaciones CRUD sobre los datos de sensores almacenados en PostgreSQL mediante ADO.NET.

El entregable son dos programas cliente que usan ADO.NET con el proveedor **Npgsql**, sin Entity Framework ni otra capa que oculte el SQL:

- **Aplicación web en C#** (ASP.NET Core, Razor Pages): la interfaz gráfica básica del módulo. Crea, consulta, actualiza y elimina sensores, lecturas, alertas y dispositivos con el **modelo conectado** (`DbCommand`, `DbDataReader`) y el **modelo desconectado** (`DbDataAdapter`, `DataTable`, `DbCommandBuilder`). Incluye un simulador de lecturas en tiempo real y una página de pruebas de rendimiento.
- **Cliente de consola en Visual Basic .NET**: usa el modelo desconectado con un `DataSet` de dos tablas relacionadas, modifica datos en memoria y los guarda con `DataAdapter.Update`, y los exporta a XML.

Todo corre en Docker (PostgreSQL, los dos clientes y Grafana), y Grafana muestra en paneles los datos procesados, incluida la auditoría de cada operación hecha desde los clientes.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-adonet

Competencia específica: manejar un lenguaje de programación visual y la conexión a la base de datos mediante código SQL embebido (clave de actividad BDM-1301T03CE01A09).

Objetivos específicos:

1. Diseñar la estructura de la base de datos PostgreSQL para almacenar datos IoT, con tablas y relaciones.
2. Configurar el entorno con Docker y establecer la conexión entre los programas cliente y la base de datos mediante ADO.NET.
3. Implementar las operaciones CRUD en el programa cliente con SQL embebido y probar su funcionalidad.
4. Integrar Grafana para visualizar los datos almacenados con paneles básicos de monitoreo.
5. Documentar el desarrollo, con el código fuente, las pruebas realizadas y los resultados obtenidos.

2. Marco teórico: ADO y ADO.NET

2.1 ADO (ActiveX Data Objects)

ADO es la tecnología de acceso a datos que Microsoft publicó en 1996 para Windows. Es un conjunto de **objetos COM** que cualquier lenguaje compatible con COM puede usar: Visual Basic 6, C++, VBScript en ASP clásico o VBA en Excel y Access. ADO no habla directamente con la base de datos: se apoya en proveedores **OLE DB** (o en ODBC a través de un puente) y ofrece una interfaz sencilla encima de ellos.

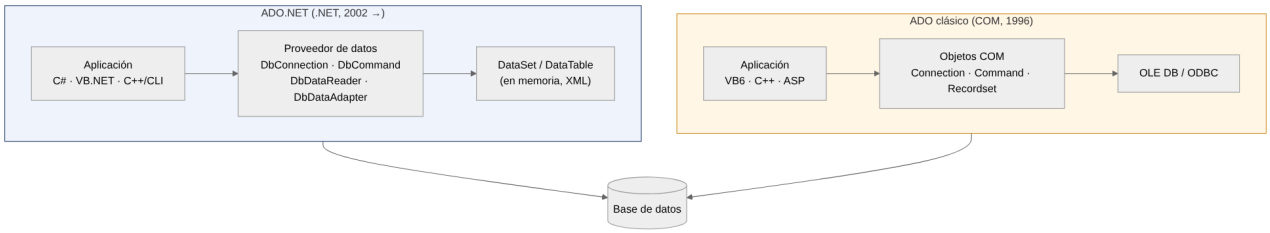
OBJETO ADO	FUNCIÓN
Connection	Abre la conexión con una cadena de conexión y administra transacciones (`BeginTrans`, `CommitTrans`, `RollbackTrans`)
Command	Ejecuta una sentencia SQL o un procedimiento almacenado, con su colección de `Parameters`
Recordset	Contiene el resultado de una consulta y permite recorrerlo, modificarlo y enviar los cambios; usa cursores del lado del servidor o del cliente
Field / Fields	Columnas de la fila actual del Recordset
Error / Errors	Errores devueltos por el proveedor

En ADO casi todo gira en torno al **Recordset**, que mantiene un cursor abierto sobre los datos. Esto es simple para aplicaciones de escritorio, pero escala mal en aplicaciones web con muchos usuarios: cada Recordset abierto ocupa una conexión y recursos en el servidor.

2.2 ADO.NET

ADO.NET llegó en 2002 con el .NET Framework. No es una nueva versión de ADO, sino un rediseño completo para aplicaciones distribuidas y web. Se divide en dos partes:

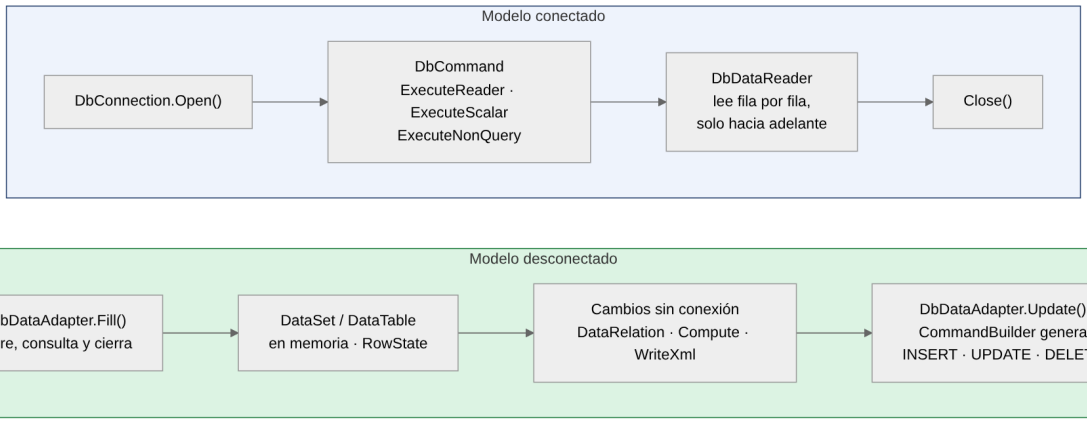
- **Proveedores de datos**: clases específicas de cada gestor que heredan de las clases base de `System.Data.Common`: `DbConnection`, `DbCommand`, `DbDataReader`, `DbDataAdapter`, `DbParameter`, `DbTransaction`. Para PostgreSQL el proveedor es Npgsql; para SQL Server, `Microsoft.Data.SqlClient`.
- **DataSet**: una base de datos en memoria, independiente del gestor, con `DataTable`, `DataRow`, `DataColumn`, `DataRelation` y restricciones. No guarda ninguna conexión abierta y se puede convertir a XML.



Arquitectura de ADO clásico frente a ADO.NET.

2.3 Modelo conectado y modelo desconectado

ADO.NET ofrece dos formas de trabajar, y en este proyecto se usan ambas:



Los dos modelos de trabajo de ADO.NET.

ASPECTO	MODELO CONECTADO	MODELO DESCONECTADO
Clases	DbConnection, DbCommand, DbDataReader	DbDataAdapter, DataSet, DataTable, DbCommandBuilder
Conexión	Abierta mientras se leen los datos	Solo durante Fill y Update
Lectura	Fila por fila, solo hacia adelante y de solo lectura	Todas las filas en memoria, en cualquier orden
Cambios	Con sentencias INSERT, UPDATE o DELETE explícitas	Se hacen en memoria; RowState marca qué filas cambiaron y Update las envía
Ventaja	Muy rápido y con poca memoria	Trabajo sin conexión, relaciones entre tablas, XML
Uso en el proyecto	Listados, detalle de sensor, CRUD de lecturas, simulador, benchmark	Pantalla de dispositivos (C#) y cliente de Visual Basic

2.4 Diferencias entre ADO y ADO.NET

ASPECTO	ADO	ADO.NET
Plataforma	COM, solo Windows	.NET; con .NET 8, Windows, Linux y macOS (en este proyecto corre en Linux dentro de Docker)
Objeto central	Recordset (una tabla con cursor)	DataSet (varias tablas relacionadas) y DataReader
Conexión	Normalmente conectada; el Recordset mantiene el cursor	Desconectada por diseño; DataReader para lecturas rápidas
Acceso al gestor	A través de OLE DB / ODBC	Proveedores nativos de cada gestor (Npgsql, SqlClient)
Relaciones entre tablas	Con JOIN en una sola consulta	DataRelation entre DataTable en memoria
Formato de intercambio	Formato binario propio (ADTG) o XML limitado	XML nativo (WriteXml, ReadXml)
Tipos de datos	VARIANT de COM	Tipos de .NET con verificación en compilación
Pool de conexiones	Del proveedor OLE DB	Integrado en cada proveedor
Estado actual	Tecnología heredada, en mantenimiento	Base de todo el acceso a datos en .NET (Entity Framework y Dapper lo usan por dentro)

2.5 Aplicación en Visual Basic, C# y C++

La misma consulta parametrizada —el umbral de un sensor— escrita en cada lenguaje muestra cómo cambia el acceso a datos según la tecnología.

Visual Basic 6 con ADO clásico. Se crean objetos COM y el resultado llega en un Recordset:

```
Dim cn As New ADODB.Connection
Dim cmd As New ADODB.Command
Dim rs As ADODB.Recordset
cn.Open "Provider=PostgreSQL OLE DB Provider;Data Source=servidor;Location=iotanalytics;"
Set cmd.ActiveConnection = cn
cmd.CommandText = "SELECT umbral_max FROM sensores WHERE codigo = ?"
cmd.Parameters.Append cmd.CreateParameter("codigo", adVarChar, adParamInput, 20, "PRE03-TEMP")
Set rs = cmd.Execute
If Not rs.EOF Then MsgBox "Umbral: " & rs!umbral_max
rs.Close: cn.Close
```

Visual Basic .NET con ADO.NET (como en el cliente `reporte-vb` de este proyecto). `Using` cierra la conexión automáticamente:

```
Using cn As New NpgsqlConnection(cadena)
    cn.Open()
    Using cmd As New NpgsqlCommand("SELECT umbral_max FROM sensores WHERE codigo = @codigo", cn)
        cmd.Parameters.AddWithValue("@codigo", "PRE03-TEMP")
        Dim umbral = CDec(cmd.ExecuteScalar())
        Console.WriteLine($"Umbral: {umbral}")
    End Using
End Using
```

C# con ADO.NET (como en la aplicación web). La sintaxis cambia, pero las clases son exactamente las mismas que en VB.NET, porque ambos lenguajes usan el mismo .NET:

```
await using var cn = new NpgsqlConnection(cadena);
await cn.OpenAsync();
await using var cmd = new NpgsqlCommand("SELECT umbral_max FROM sensores WHERE codigo = @codigo",
cn);
cmd.Parameters.AddWithValue("@codigo", "PRE03-TEMP");
var umbral = (decimal)(await cmd.ExecuteScalarAsync())!;
```

C++ nativo con ADO clásico. C++ usa ADO a través de COM con la directiva `#import`, que genera clases con punteros inteligentes como `_ConnectionPtr`:

```
#import "msado15.dll" no_namespace rename("EOF", "EndOfFile")
CoInitialize(NULL);
_ConnectionPtr cn(__uuidof(Connection));
_CommandPtr cmd(__uuidof(Command));
cn->Open(L"Provider=PostgreSQL OLE DB Provider;Data Source=servidor;Location=iotanalytics;", L"",
L"", adConnectUnspecified);
cmd->ActiveConnection = cn;
cmd->CommandText = L"SELECT umbral_max FROM sensores WHERE codigo = ?";
cmd->Parameters->Append(cmd->CreateParameter(L"codigo", adVarChar, adParamInput, 20, L"PRE03-
TEMP"));
_RecordsetPtr rs = cmd->Execute(NULL, NULL, adCmdText);
if (!rs->EndOfFile) wprintf(L"Umbral: %s\n", (LPCWSTR)(_bstr_t)rs->Fields->GetItem(L"umbral_max")-
>Value);
rs->Close(); cn->Close(); CoUninitialize();
```

C++/CLI con ADO.NET. El dialecto C++/CLI de Microsoft compila a .NET, así que puede usar las clases de ADO.NET con la sintaxis ^ para objetos administrados:

```
NpgsqlConnection^ cn = gcnew NpgsqlConnection(cadena);
cn->Open();
NpgsqlCommand^ cmd = gcnew NpgsqlCommand("SELECT umbral_max FROM sensores WHERE codigo = @codigo",
cn);
cmd->Parameters->AddWithValue("@codigo", "PRE03-TEMP");
Decimal umbral = safe_cast<Decimal>(cmd->ExecuteScalar());
cn->Close();
```

LENGUAJE	TECNOLOGÍA HABITUAL	CARACTERÍSTICAS
Visual Basic 6	ADO (COM)	Objetos ADODB; Recordset con cursor; solo Windows
Visual Basic .NET	ADO.NET	Mismas clases que C#; Using, Option Strict, LINQ; usado en el cliente de este proyecto
C#	ADO.NET	Lenguaje principal de .NET; async/await; usado en la aplicación web de este proyecto
C++ nativo	ADO por COM (#import) u ODBC	Control total de la memoria; más código; solo Windows para ADO
C++/CLI	ADO.NET	Puente entre código C++ nativo y .NET; solo Windows

Para un sistema nuevo en NexoDev la elección natural es **ADO.NET con C#**: es multiplataforma, corre en contenedores Linux y tiene proveedores nativos para PostgreSQL. ADO clásico se mantiene en aplicaciones heredadas de Windows, y C++ con ADO u ODBC se usa cuando se necesita integrar un sistema nativo existente. La Tarea 10 de esta materia mostró la alternativa en C puro: SQL embebido con ECPG.

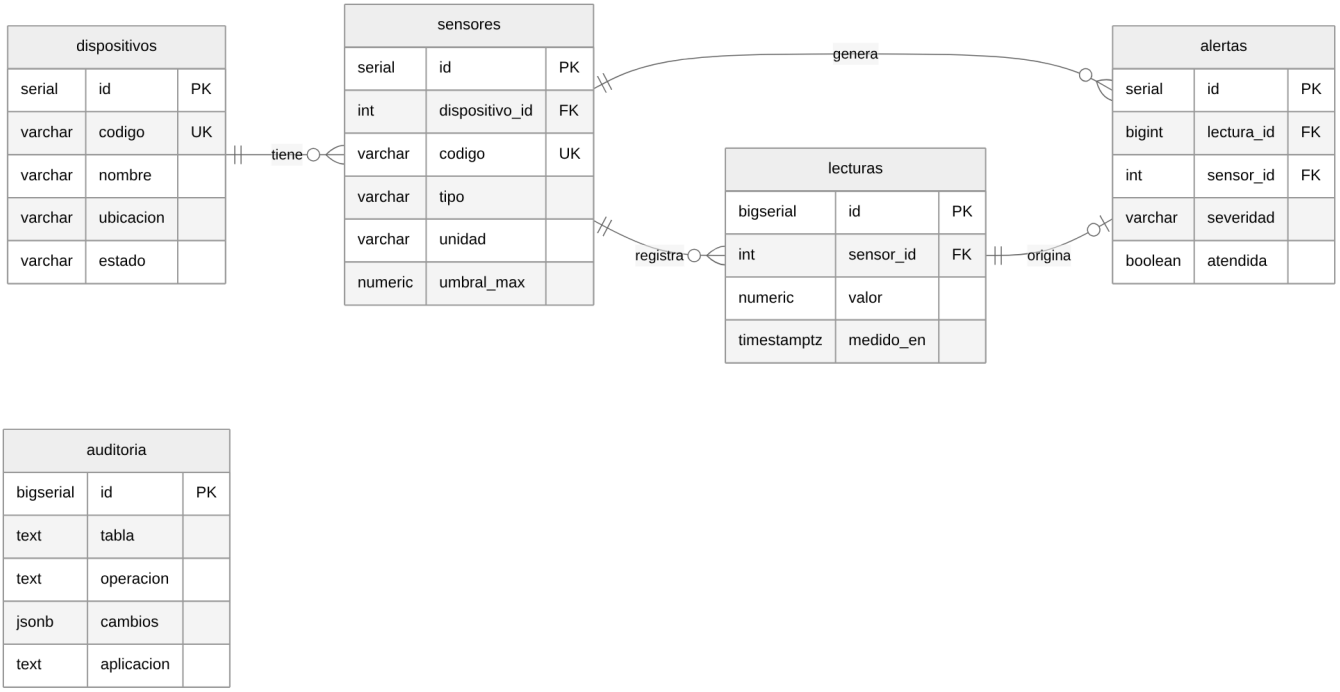
2.6 Conceptos clave usados en el proyecto

CONCEPTO	PARA QUÉ SIRVE	DÓNDE SE USA
Cadena de conexión	Servidor, base, usuario, SSL y nombre de la aplicación	`docker-compose.yml` (variable `ConnectionStrings__Iot`)
DbProviderFactory	Crea conexiones, comandos y adaptadores sin depender del proveedor concreto	`Program.cs`, `ConexionIot.cs`, `Program.vb`
Pool de conexiones	Reutiliza conexiones físicas: abrir y cerrar es barato	Automático en Npgsql
Parámetros (@nombre)	Envían los valores separados del SQL: evitan la inyección SQL	Todas las consultas
ExecuteReader / ExecuteScalar / ExecuteNonQuery	Leer filas / un solo valor / filas afectadas	`RepositorioIot.cs`
DbTransaction	Agrupa operaciones: todo o nada	Registrar lectura, actualizar umbral, eliminar sensor, guardar dispositivos
Prepare	El servidor analiza el SQL una vez y lo reutiliza	Simulador y benchmark
DbBatch	Varias sentencias en un solo viaje de red (.NET 6+)	Benchmark
DbCommandBuilder	Genera el INSERT, UPDATE y DELETE de un DataAdapter	Dispositivos (C#) y ajustar (VB)
Concurrencia optimista	El UPDATE generado compara los valores originales; si otro usuario cambió la fila, falla con DBConcurrencyException	`GuardarEstadosAsync`
DbException.SqlState	Código estándar del error para tratarlo según su tipo	Método `Ejecutar` del repositorio

3. Diseño

3.1 Base de datos

Cinco tablas: las cuatro relacionadas del proyecto IoT y una de auditoría. Las restricciones `UNIQUE`, `CHECK` y las llaves foráneas protegen la integridad, y dos disparadores automatizan tareas que el cliente no tiene que programar: crear las alertas y registrar en la auditoría cada alta, cambio o baja de sensores y dispositivos, junto con el nombre de la aplicación que la hizo (`application_name` de la cadena de conexión).



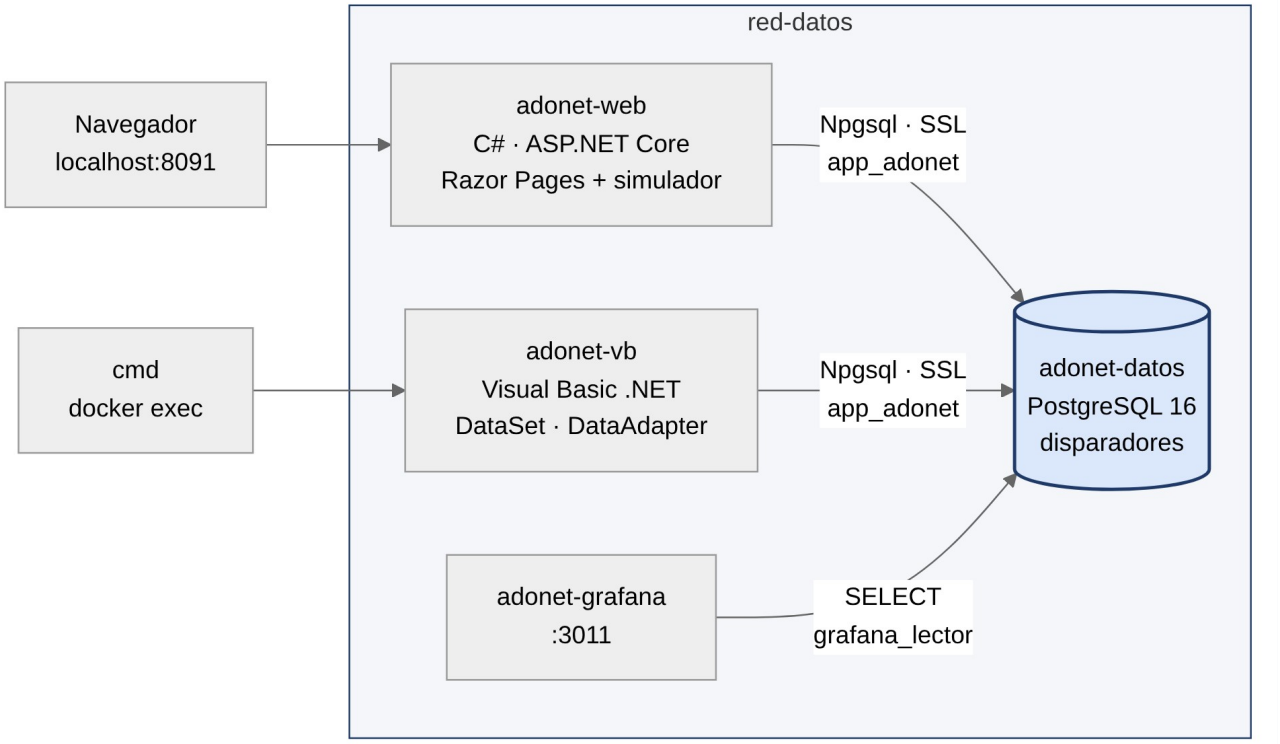
Modelo entidad-relación.

USUARIO	PERMISOS
`app_adonet`	SELECT, INSERT, UPDATE y DELETE sobre las tablas de datos; solo lectura de la auditoría; no puede cambiar la estructura
`grafana_lector`	Solo SELECT; zona horaria de México para los paneles

3.2 Arquitectura y conexión

Cuatro contenedores en una red interna. La base de datos no expone puertos a la PC; los dos clientes se conectan con Npgsql por SSL usando la misma cadena de conexión, que solo cambia en `Application Name` para que la auditoría distinga quién hizo cada cambio:

```
Host=datos;Database=iotanalytics;Username=app_adonet;Password=adonet123;
SSL Mode=Require;Application Name=web-csharp;Timezone=America/Mexico_City
```

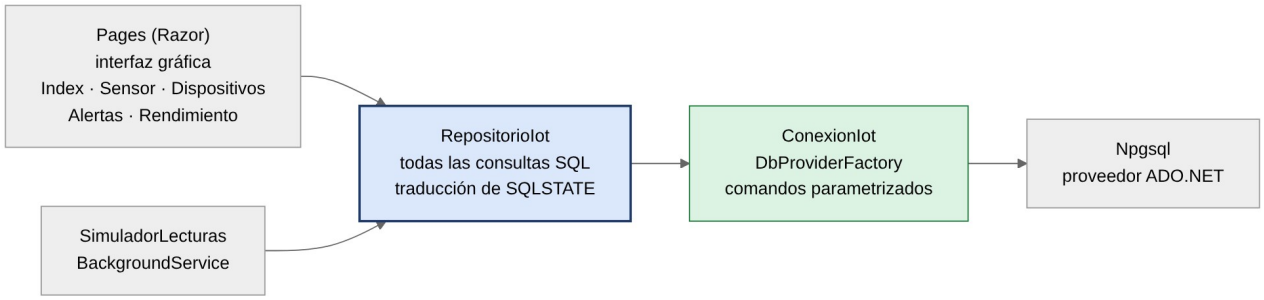


Arquitectura del entorno con Docker.

CONTENEDOR	IMAGEN	PUERTO EN LA PC
adonet-datos	postgres:16 (SSL)	Ninguno
adonet-web	ASP.NET Core 8 (compilada en Docker con el SDK de .NET 8)	8091
adonet-vb	Runtime de .NET 8 con el programa de Visual Basic	Ninguno (`docker exec`)
adonet-grafana	grafana/grafana:11.3.0	3011

3.3 Estructura del cliente en C#

El código se separa en capas para que quede limpio y cada parte tenga una sola responsabilidad: las páginas solo reciben datos del usuario y muestran resultados; el repositorio concentra todo el SQL y el manejo de errores; la clase de conexión crea objetos con la fábrica del proveedor.



Capas del cliente ADO.NET en C#.

3.4 Operaciones CRUD

OPERACIÓN	PANTALLA O COMANDO	TÉCNICA DE ADO.NET
Crear	Nuevo sensor	`INSERT ... RETURNING id` con `ExecuteScalar`; la unidad

OPERACIÓN	PANTALLA O COMANDO	TÉCNICA DE ADO.NET
		se deriva del tipo
Crear	Registrar lectura	`INSERT` + consulta de la alerta en una `DbTransaction`
Crear	Simulador (segundo plano)	Comando preparado (`Prepare`) y una transacción por ronda
Leer	Sensores, detalle, alertas, auditoría	`ExecuteReader` + `DbDataReader`; `IsDBNull` para valores nulos
Leer	Reporte VB	`DataAdapter.Fill` de dos tablas en un `DataSet` + `DataRelation`
Actualizar	Umbral	`SELECT ... FOR UPDATE` y `UPDATE` con `ExecuteNonQuery` en una transacción
Actualizar	Estado de dispositivos (C#) y umbrales por tipo (VB)	`DataTable` modificada en memoria + `DataAdapter.Update` con `CommandBuilder`
Actualizar	Atender alertas	`UPDATE` con `ExecuteNonQuery`, que devuelve las filas afectadas
Eliminar	Lectura	`DELETE` (la alerta se borra en cascada)
Eliminar	Sensor / sensor con lecturas	Sin opción la llave foránea lo impide; con opción, tres `DELETE` en una transacción

4. Implementación

4.1 Estructura del proyecto

```
iot-adonet/
├── docker-compose.yml              → base de datos, web (C#), vb y Grafana
├── datos/init.sql                  → tablas, disparadores, usuarios y datos de prueba
├── web/                            → CLIENTE ADO.NET EN C#
│   ├── IotAdoNet.csproj            → ASP.NET Core 8 + Npgsql
│   ├── Program.cs                  → registro del proveedor y servicios
│   ├── Datos/ConexionIot.cs        → fábrica, conexión y comandos parametrizados
│   ├── Datos/RepositorioIot.cs     → todas las operaciones CRUD
│   ├── Datos/Modelos.cs            → registros de datos y errores
│   ├── Servicios/SimuladorLecturas.cs → lecturas en tiempo real
│   ├── Pages/                      → interfaz gráfica (Razor Pages)
│   └── Dockerfile
├── vb/                             → CLIENTE ADO.NET EN VISUAL BASIC
│   ├── ReporteVB.vbproj · Program.vb · Dockerfile
├── grafana/                        → origen de datos y tablero
└── docs/Documentacion_Tarea11.pdf
```

4.2 Buenas prácticas aplicadas

- **Solo SQL parametrizado:** ningún valor del usuario se concatena al texto de una consulta.
- **Liberación de recursos:** `await using` (C#) y `Using` (VB) cierran conexiones, comandos y lectores aunque ocurra un error, y la conexión vuelve al pool.
- **Transacciones** en toda operación de varios pasos, con `Rollback` explícito ante un error.
- **Errores traducidos por SQLSTATE:** el usuario ve un mensaje claro y el código estándar (23505, 23503, 23514), no una excepción técnica.
- **Independencia del proveedor:** el código trabaja con `DbConnection`, `DbCommand` y `DbProviderFactory`; cambiar de gestor solo cambia el registro del proveedor.

- **Programación asíncrona** en la web (`OpenAsync`, `ExecuteReaderAsync`) para no bloquear hilos mientras se espera a la base de datos.
- **Configuración fuera del código:** la cadena de conexión llega por variable de entorno.
- **Datos válidos por diseño:** la unidad de un sensor se deriva de su tipo y no se captura a mano.

4.3 Registro del proveedor: Program.cs

```
// ====================================================================
// TAREA 11 · Programacion del lado del cliente: ADO.NET con C#
// IoT Analytics Dashboard (NexoDev Solutions)
// Aplicacion web (interfaz grafica basica) que realiza operaciones CRUD
// sobre PostgreSQL usando ADO.NET puro: DbConnection, DbCommand,
// DbDataReader, DbDataAdapter y DbTransaction. Sin Entity Framework.
// ====================================================================
using System.Data.Common;
using IotAdoNet.Datos;
using IotAdoNet.Servicios;
using Npgsql;

var builder = WebApplication.CreateBuilder(args);

// Modelo de proveedores de ADO.NET: se registra el proveedor de PostgreSQL una sola vez
// y el resto del programa trabaja con las clases base de System.Data.Common.
DbProviderFactories.RegisterFactory("Npgsql", NpgsqlFactory.Instance);
var fabrica = DbProviderFactories.GetFactory("Npgsql");

var cadena = builder.Configuration.GetConnectionString("Iot")
             ?? throw new InvalidOperationException("Falta la cadena de conexion
ConnectionStrings__Iot");

builder.Services.AddSingleton(new ConexionIot(fabrica, cadena));
builder.Services.AddScoped<RepositorioIot>();
builder.Services.AddHostedService<SimuladorLecturas>();
builder.Services.AddRazorPages();

var app = builder.Build();
app.UseStaticFiles();
app.MapRazorPages();
app.Run();
```

4.4 Conexión y comandos: ConexionIot.cs

```
using System.Data.Common;

namespace IotAdoNet.Datos;

/// <summary>
/// Crea y abre conexiones con la fabrica del proveedor. El pool de conexiones
/// de ADO.NET reutiliza las conexiones fisicas: abrir y cerrar es barato.
/// </summary>
public sealed class ConexionIot(DbProviderFactory fabrica, string cadena)
{
    public DbProviderFactory Fabrica { get; } = fabrica;

    public async Task<DbConnection> AbrirAsync(CancellationToken ct = default)
    {
        var conexion = Fabrica.CreateConnection()!;
        conexion.ConnectionString = cadena;
        await conexion.OpenAsync(ct);
        return conexion;
    }

    /// <summary>Crea un comando con parametros nombrados (@nombre): nunca se concatena
texto.</summary>
```

```
    public static DbCommand Comando(DbConnection conexion, string sql, DbTransaction? tx = null,
                                     params (string Nombre, object? Valor)[] parametros)
    {
        var cmd = conexion.CreateCommand();
        cmd.CommandText = sql;
        cmd.Transaction = tx;
        foreach (var (nombre, valor) in parametros)
        {
            var p = cmd.CreateParameter();
            p.ParameterName = nombre;
            p.Value = valor ?? DBNull.Value;
            cmd.Parameters.Add(p);
        }
        return cmd;
    }
}
```

4.5 Operaciones CRUD: RepositorioIot.cs

```
using System.Data;
using System.Data.Common;
using System.Diagnostics;

namespace IotAdoNet.Datos;

/// <summary>
/// Acceso a datos con ADO.NET. Usa los dos modelos de ADO.NET:
///  - Conectado: DbCommand + DbDataReader (lectura rapida, hacia adelante).
///  - Desconectado: DbDataAdapter + DataTable + DbCommandBuilder (ver dispositivos).
/// Todas las consultas usan parametros (@nombre): el valor viaja aparte del SQL.
/// </summary>
public sealed class RepositorioIot(ConexionIot bd)
{
    // ============================ LEER ============================

    public async Task<Resumen> ResumenAsync()
    {
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT (SELECT COUNT(*) FROM lecturas),
                   (SELECT COUNT(*) FROM lecturas WHERE medido_en > NOW() - INTERVAL '1 minute'),
                   (SELECT COUNT(*) FROM alertas WHERE NOT atendida),
                   (SELECT COUNT(*) FROM alertas WHERE NOT atendida AND severidad = 'critica'),
                   (SELECT COUNT(*) FROM dispositivos WHERE estado = 'activo'),
                   (SELECT COUNT(*) FROM sensores)
            """);
        await using var lector = await cmd.ExecuteReaderAsync();
        await lector.ReadAsync();
        return new Resumen(lector.GetInt64(0), lector.GetInt64(1), lector.GetInt64(2),
                           lector.GetInt64(3), lector.GetInt64(4), lector.GetInt64(5));
    }

    /// <summary>Sensores con su ultima lectura (DataReader: modelo conectado).</summary>
    public async Task<List<SensorFila>> SensoresAsync()
    {
        var lista = new List<SensorFila>();
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT s.id, s.codigo, s.tipo, s.unidad, s.umbral_max, d.codigo, d.estado,
                   u.valor, to_char(u.medido_en, 'HH24:MI:SS')
            FROM sensores s
            JOIN dispositivos d ON d.id = s.dispositivo_id
            LEFT JOIN LATERAL (SELECT l.valor, l.medido_en FROM lecturas l
                               WHERE l.sensor_id = s.id
                               ORDER BY l.medido_en DESC LIMIT 1) u ON TRUE
            ORDER BY s.codigo
            """);
```

```
        await using var lector = await cmd.ExecuteReaderAsync();
        while (await lector.ReadAsync())
        {
            lista.Add(new SensorFila(
                lector.GetInt32(0), lector.GetString(1), lector.GetString(2), lector.GetString(3),
                lector.GetDecimal(4), lector.GetString(5), lector.GetString(6),
                lector.IsDBNull(7) ? null : lector.GetDecimal(7),     // NULL si no tiene lecturas
                lector.IsDBNull(8) ? null : lector.GetString(8)));
        }
        return lista;
    }

    public async Task<SensorFila?> SensorAsync(string codigo)
        => (await SensoresAsync()).FirstOrDefault(s => s.Codigo == codigo);

    public async Task<List<LecturaFila>> LecturasAsync(string codigo, int cuantas = 20)
    {
        var lista = new List<LecturaFila>();
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT l.id, l.valor, to_char(l.medido_en, 'YYYY-MM-DD HH24:MI:SS'), a.severidad
            FROM lecturas l
            JOIN sensores s ON s.id = l.sensor_id
            LEFT JOIN alertas a ON a.lectura_id = l.id
            WHERE s.codigo = @codigo
            ORDER BY l.medido_en DESC
            LIMIT @cuantas
            """, null, ("@codigo", codigo), ("@cuantas", cuantas));
        await using var lector = await cmd.ExecuteReaderAsync();
        while (await lector.ReadAsync())
            lista.Add(new LecturaFila(lector.GetInt64(0), lector.GetDecimal(1),
lector.GetString(2),
                                      lector.IsDBNull(3) ? null : lector.GetString(3)));
        return lista;
    }

    public async Task<List<DispositivoOpcion>> DispositivosOpcionesAsync()
    {
        var lista = new List<DispositivoOpcion>();
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, "SELECT codigo, nombre FROM dispositivos
ORDER BY codigo");
        await using var lector = await cmd.ExecuteReaderAsync();
        while (await lector.ReadAsync())
            lista.Add(new DispositivoOpcion(lector.GetString(0), lector.GetString(1)));
        return lista;
    }

    public async Task<List<AlertaFila>> AlertasPendientesAsync()
    {
        var lista = new List<AlertaFila>();
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT a.id, to_char(a.creada_en, 'HH24:MI:SS'), s.codigo, d.codigo, a.severidad,
a.valor, a.umbral
            FROM alertas a
            JOIN sensores s ON s.id = a.sensor_id
            JOIN dispositivos d ON d.id = s.dispositivo_id
            WHERE NOT a.atendida
            ORDER BY (a.severidad = 'critica') DESC, a.creada_en DESC
            LIMIT 50
            """);
        await using var lector = await cmd.ExecuteReaderAsync();
        while (await lector.ReadAsync())
            lista.Add(new AlertaFila(lector.GetInt32(0), lector.GetString(1), lector.GetString(2),
                                     lector.GetString(3), lector.GetString(4),
lector.GetDecimal(5),
                                     lector.GetDecimal(6)));
```

```
        return lista;
    }

    /// <summary>Ultimos cambios registrados por el disparador de auditoria.</summary>
    public async Task<List<(string Hora, string Tabla, string Operacion, string Registro, string
Cambios, string App)>> AuditoriaAsync()
    {
        var lista = new List<(string, string, string, string, string, string)>();
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT to_char(ocurrido_en, 'HH24:MI:SS'), tabla, operacion, COALESCE(registro, ''),
                   CASE WHEN operacion = 'UPDATE' THEN cambios::text ELSE '' END, aplicacion
            FROM auditoria ORDER BY id DESC LIMIT 8
            """);
        await using var lector = await cmd.ExecuteReaderAsync();
        while (await lector.ReadAsync())
            lista.Add((lector.GetString(0), lector.GetString(1), lector.GetString(2),
                       lector.GetString(3), lector.GetString(4), lector.GetString(5)));
        return lista;
    }

    // ============================ CREAR ============================

    /// <summary>Cada tipo de sensor tiene una sola unidad valida: no se captura a mano.</summary>
    public static readonly IReadOnlyDictionary<string, string> UnidadPorTipo = new
Dictionary<string, string>
    {
        ["temperatura"] = "°C", ["vibracion"] = "mm/s", ["energia"] = "kWh", ["humedad"] = "%"
    };

    /// <summary>INSERT ... RETURNING con ExecuteScalar: devuelve el id generado.</summary>
    public async Task<int> CrearSensorAsync(string codigo, string dispositivo, string tipo,
decimal umbral)
    {
        if (!UnidadPorTipo.TryGetValue(tipo, out var unidad))
            throw new ErrorDatos($"El tipo {tipo} no es valido.", null);
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, """
            INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max)
            SELECT d.id, @codigo, @tipo, @unidad, @umbral FROM dispositivos d WHERE d.codigo =
@dispositivo
            RETURNING id
            """, null, ("@codigo", codigo.Trim().ToUpperInvariant()), ("@tipo", tipo),
                       ("@unidad", unidad), ("@umbral", umbral), ("@dispositivo", dispositivo));
        var id = await Ejecutar(() => cmd.ExecuteScalarAsync());
        return id is null ? throw new ErrorDatos($"El dispositivo {dispositivo} no existe.", null)
: Convert.ToInt32(id);
    }

    /// <summary>Inserta la lectura y consulta si el disparador genero una alerta, en una
transaccion.</summary>
    public async Task<(long Id, string Situacion)> RegistrarLecturaAsync(string codigo, decimal
valor)
    {
        await using var cn = await bd.AbrirAsync();
        await using var tx = await cn.BeginTransactionAsync();
        await using var ins = ConexionIot.Comando(cn, """
            INSERT INTO lecturas (sensor_id, valor)
            SELECT id, @valor FROM sensores WHERE codigo = @codigo
            RETURNING id
            """, tx, ("@valor", valor), ("@codigo", codigo));
        var id = await Ejecutar(() => ins.ExecuteScalarAsync());
        if (id is null)
        {
            await tx.RollbackAsync();
            throw new ErrorDatos($"El sensor {codigo} no existe.", null);
        }
```

```
        await using var sev = ConexionIot.Comando(cn, "SELECT severidad FROM alertas WHERE
lectura_id = @id",
                                                  tx, ("@id", Convert.ToInt64(id)));
        var severidad = await sev.ExecuteScalarAsync();
        await tx.CommitAsync();
        return (Convert.ToInt64(id), severidad as string ?? "normal");
    }

    // ============================ ACTUALIZAR ============================

    /// <summary>UPDATE con ExecuteNonQuery: devuelve el numero de filas afectadas.</summary>
    public async Task<decimal> ActualizarUmbralAsync(string codigo, decimal umbral)
    {
        await using var cn = await bd.AbrirAsync();
        await using var tx = await cn.BeginTransactionAsync();
        await using var leer = ConexionIot.Comando(cn,
            "SELECT umbral_max FROM sensores WHERE codigo = @codigo FOR UPDATE", tx, ("@codigo",
codigo));
        var anterior = await leer.ExecuteScalarAsync();
        if (anterior is null)
            throw new ErrorDatos($"El sensor {codigo} no existe.", null);

        await using var upd = ConexionIot.Comando(cn,
            "UPDATE sensores SET umbral_max = @umbral WHERE codigo = @codigo", tx,
            ("@umbral", umbral), ("@codigo", codigo));
        await Ejecutar(() => upd.ExecuteNonQueryAsync());
        await tx.CommitAsync();
        return Convert.ToDecimal(anterior);
    }

    public async Task<int> AtenderAlertaAsync(int id)
    {
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn,
            "UPDATE alertas SET atendida = TRUE WHERE id = @id AND NOT atendida", null, ("@id",
id));
        return await cmd.ExecuteNonQueryAsync();
    }

    public async Task<int> AtenderTodasAsync()
    {
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, "UPDATE alertas SET atendida = TRUE WHERE
NOT atendida");
        return await cmd.ExecuteNonQueryAsync();
    }

    // ---------- Modelo desconectado: DataAdapter + DataTable + CommandBuilder ----------

    /// <summary>Llena un DataTable con los dispositivos; la conexion se cierra al terminar
Fill.</summary>
    public async Task<DataTable> DispositivosTablaAsync()
    {
        await using var cn = await bd.AbrirAsync();
        using var adaptador = bd.Fabrica.CreateDataAdapter()!;
        adaptador.SelectCommand = ConexionIot.Comando(cn,
            "SELECT id, codigo, nombre, ubicacion, estado FROM dispositivos ORDER BY codigo");
        var tabla = new DataTable("dispositivos");
        adaptador.Fill(tabla);
        return tabla;
    }

    /// <summary>
    /// Modifica las filas en memoria y las guarda con DataAdapter.Update. El CommandBuilder
    /// genera el UPDATE a partir del SELECT; solo se envian las filas con RowState = Modified.
    /// </summary>
    public async Task<(int Modificadas, string SqlGenerado)>
GuardarEstadosAsync(IDictionary<string, string> nuevos)
```

```
    {
        await using var cn = await bd.AbrirAsync();
        await using var tx = await cn.BeginTransactionAsync();
        using var adaptador = bd.Fabrica.CreateDataAdapter()!;
        adaptador.SelectCommand = ConexionIot.Comando(cn,
            "SELECT id, codigo, nombre, ubicacion, estado FROM dispositivos ORDER BY codigo", tx);
        using var constructor = bd.Fabrica.CreateCommandBuilder()!;
        constructor.DataAdapter = adaptador;

        var tabla = new DataTable("dispositivos");
        adaptador.Fill(tabla);
        foreach (DataRow fila in tabla.Rows)
        {
            var codigo = (string)fila["codigo"];
            if (nuevos.TryGetValue(codigo, out var estado) && estado != (string)fila["estado"])
                fila["estado"] = estado;                        // RowState pasa a Modified
        }

        var actualizar = constructor.GetUpdateCommand();
        actualizar.Transaction = tx;
        adaptador.UpdateCommand = actualizar;
        try
        {
            var n = await Ejecutar(() => Task.FromResult(adaptador.Update(tabla)));
            await tx.CommitAsync();
            return (n, actualizar.CommandText);
        }
        catch (DBConcurrencyException)
        {
            // el UPDATE generado compara los valores originales: si otro usuario cambio la fila,
falla
            await tx.RollbackAsync();
            throw new ErrorDatos("Otro usuario modifico el dispositivo mientras lo editabas.
Recarga la pagina.", null);
        }
    }

    // ============================ ELIMINAR ============================

    public async Task<int> EliminarLecturaAsync(long id)
    {
        await using var cn = await bd.AbrirAsync();
        await using var cmd = ConexionIot.Comando(cn, "DELETE FROM lecturas WHERE id = @id", null,
("@id", id));
        return await Ejecutar(() => cmd.ExecuteNonQueryAsync());
    }

    /// <summary>
    /// Sin conLecturas, la llave foranea rechaza el borrado si hay lecturas (23503).
    /// Con conLecturas, borra alertas, lecturas y sensor en UNA transaccion: todo o nada.
    /// </summary>
    public async Task<(int Lecturas, int Alertas)> EliminarSensorAsync(string codigo, bool
conLecturas)
    {
        await using var cn = await bd.AbrirAsync();
        await using var tx = await cn.BeginTransactionAsync();
        int lecturas = 0, alertas = 0;
        try
        {
            if (conLecturas)
            {
                await using var a = ConexionIot.Comando(cn,
                    "DELETE FROM alertas WHERE sensor_id = (SELECT id FROM sensores WHERE codigo =
@c)", tx, ("@c", codigo));
                alertas = await a.ExecuteNonQueryAsync();
                await using var l = ConexionIot.Comando(cn,
                    "DELETE FROM lecturas WHERE sensor_id = (SELECT id FROM sensores WHERE codigo
= @c)", tx, ("@c", codigo));
```

```
                lecturas = await l.ExecuteNonQueryAsync();
            }
            await using var s = ConexionIot.Comando(cn, "DELETE FROM sensores WHERE codigo = @c",
tx, ("@c", codigo));
            var n = await Ejecutar(() => s.ExecuteNonQueryAsync());
            if (n == 0) throw new ErrorDatos($"El sensor {codigo} no existe.", null);
            await tx.CommitAsync();
            return (lecturas, alertas);
        }
        catch
        {
            await tx.RollbackAsync();
            throw;
        }
    }

    // ============================ RENDIMIENTO ============================

    /// <summary>
    /// Inserta n lecturas de cuatro formas y las elimina al final:
    ///  A. un comando por lectura, cada uno con su propia transaccion (autocommit)
    ///  B. un comando por lectura dentro de una transaccion
    ///  C. comando preparado (Prepare) reutilizado dentro de una transaccion
    ///  D. DbBatch: todas las sentencias en un solo viaje al servidor
    /// </summary>
    public async Task<List<(string Forma, double Ms)>> BenchmarkAsync(int n)
    {
        const string sql = "INSERT INTO lecturas (sensor_id, valor) SELECT id, @valor FROM
sensores WHERE codigo = 'PRE03-VIB'";
        var resultados = new List<(string, double)>();
        await using var cn = await bd.AbrirAsync();
        await using var max = ConexionIot.Comando(cn, "SELECT COALESCE(MAX(id), 0) FROM
lecturas");
        var desde = Convert.ToInt64(await max.ExecuteScalarAsync());
        var reloj = Stopwatch.StartNew();

        for (var i = 0; i < n; i++)
        {
            await using var cmd = ConexionIot.Comando(cn, sql, null, ("@valor", 5m + i % 10 /
10m));
            await cmd.ExecuteNonQueryAsync();
        }
        resultados.Add(("A. Un comando por lectura (autocommit)",
reloj.Elapsed.TotalMilliseconds));

        reloj.Restart();
        await using (var tx = await cn.BeginTransactionAsync())
        {
            for (var i = 0; i < n; i++)
            {
                await using var cmd = ConexionIot.Comando(cn, sql, tx, ("@valor", 5m + i % 10 /
10m));
                await cmd.ExecuteNonQueryAsync();
            }
            await tx.CommitAsync();
        }
        resultados.Add(("B. Un comando por lectura, una transaccion",
reloj.Elapsed.TotalMilliseconds));

        reloj.Restart();
        await using (var tx = await cn.BeginTransactionAsync())
        {
            await using var cmd = ConexionIot.Comando(cn, sql, tx, ("@valor", 5m));
            await cmd.PrepareAsync();                   // el servidor analiza el SQL una sola vez
            for (var i = 0; i < n; i++)
            {
                cmd.Parameters[0].Value = 5m + i % 10 / 10m;
                await cmd.ExecuteNonQueryAsync();
```

```
            }
            await tx.CommitAsync();
        }
        resultados.Add(("C. Comando preparado, una transaccion",
reloj.Elapsed.TotalMilliseconds));

        reloj.Restart();
        if (bd.Fabrica.CanCreateBatch)
        {
            await using var lote = cn.CreateBatch();
            for (var i = 0; i < n; i++)
            {
                var bc = lote.CreateBatchCommand();
                bc.CommandText = sql;
                var p = bc.CreateParameter();
                p.ParameterName = "@valor";
                p.Value = 5m + i % 10 / 10m;
                bc.Parameters.Add(p);
                lote.BatchCommands.Add(bc);
            }
            await lote.ExecuteNonQueryAsync();          // un solo viaje de red, una transaccion
implicita
            resultados.Add(("D. DbBatch (un solo viaje al servidor)",
reloj.Elapsed.TotalMilliseconds));
        }

        await using var limpiar = ConexionIot.Comando(cn,
            "DELETE FROM lecturas WHERE id > @desde AND sensor_id = (SELECT id FROM sensores WHERE
codigo = 'PRE03-VIB')",
            null, ("@desde", desde));
        await limpiar.ExecuteNonQueryAsync();
        return resultados;
    }

    // ============================ ERRORES ============================

    /// <summary>Ejecuta una operacion y traduce los errores de PostgreSQL por su
SQLSTATE.</summary>
    private static async Task<T> Ejecutar<T>(Func<Task<T>> operacion)
    {
        try
        {
            return await operacion();
        }
        catch (DbException ex)
        {
            var mensaje = ex.SqlState switch
            {
                "23505" => "Ya existe un registro con ese codigo.",
                "23503" => "El registro tiene datos que dependen de el (lecturas o alertas).",
                "23514" => "El valor no cumple una regla de la tabla (CHECK): revisa el tipo, el
umbral o el estado.",
                "22P02" => "El formato del dato no es valido.",
                _ => "Error de base de datos."
            };
            throw new ErrorDatos($"{mensaje} [SQLSTATE {ex.SqlState}]", ex.SqlState);
        }
    }
}
```

4.6 Simulador de lecturas

```
using System.Data.Common;
using IotAdoNet.Datos;

namespace IotAdoNet.Servicios;
```

```
/// <summary>
/// Servicio en segundo plano que envia una lectura por sensor cada segundo, como lo
/// harian los dispositivos IoT. Usa un comando preparado dentro de una transaccion
/// por ronda (la forma mas eficiente segun la prueba de rendimiento).
/// </summary>
public sealed class SimuladorLecturas(ConexionIot bd, IConfiguration config,
ILogger<SimuladorLecturas> log)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        if (!config.GetValue("SIMULADOR", true)) return;
        var azar = new Random();
        var rondas = 0;
        long total = 0;
        var sensores = new List<(string Codigo, decimal Umbral)>();

        while (!ct.IsCancellationRequested)
        {
            try
            {
                await using var cn = await bd.AbrirAsync(ct);
                if (rondas % 30 == 0) sensores = await CargarSensoresAsync(cn, ct);   // incluye
sensores nuevos

                await using var tx = await cn.BeginTransactionAsync(ct);
                await using var cmd = ConexionIot.Comando(cn,
                    "INSERT INTO lecturas (sensor_id, valor) SELECT id, @valor FROM sensores WHERE
codigo = @codigo",
                    tx, ("@valor", 0m), ("@codigo", ""));
                await cmd.PrepareAsync(ct);
                foreach (var (codigo, umbral) in sensores)
                {
                    var r = azar.NextDouble();
                    var valor = umbral * (decimal)(0.76 + 0.08 * azar.NextDouble());
                    if (r < 0.02) valor = umbral * 1.10m;          // pico: advertencia
                    else if (r < 0.03) valor = umbral * 1.30m;     // pico: critica
                    cmd.Parameters[0].Value = Math.Round(valor, 2);
                    cmd.Parameters[1].Value = codigo;
                    total += await cmd.ExecuteNonQueryAsync(ct);
                }
                await tx.CommitAsync(ct);
                if (++rondas % 60 == 0)
                    log.LogInformation("Simulador: {Rondas} rondas, {Total} lecturas registradas",
rondas, total);
            }
            catch (Exception ex) when (ex is DbException or TimeoutException)
            {
                log.LogWarning("Simulador: {Mensaje}. Reintentando...", ex.Message);
            }
            await Task.Delay(1000, ct);
        }
    }

    private static async Task<List<(string, decimal)>> CargarSensoresAsync(DbConnection cn,
CancellationToken ct)
    {
        var lista = new List<(string, decimal)>();
        await using var cmd = ConexionIot.Comando(cn, """
            SELECT s.codigo, s.umbral_max FROM sensores s
            JOIN dispositivos d ON d.id = s.dispositivo_id
            WHERE d.estado = 'activo' ORDER BY s.id
            """);
        await using var lector = await cmd.ExecuteReaderAsync(ct);
        while (await lector.ReadAsync(ct))
            lista.Add((lector.GetString(0), lector.GetDecimal(1)));
        return lista;
    }
```

```
}
```

4.7 Interfaz gráfica: ejemplo de página

Cada página de Razor tiene un modelo en C# que solo llama al repositorio y muestra el resultado. Por ejemplo, la administración de un sensor:

```
using IotAdoNet.Datos;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace IotAdoNet.Pages;

public class SensorModel(RepositorioIot repo) : PageModel
{
    public SensorFila? Sensor { get; private set; }
    public List<LecturaFila> Lecturas { get; private set; } = [];

    public async Task<IActionResult> OnGetAsync(string codigo)
    {
        Sensor = await repo.SensorAsync(codigo);
        if (Sensor is null)
        {
            TempData["Error"] = $"El sensor {codigo} no existe.";
            return RedirectToPage("/Index");
        }
        Lecturas = await repo.LecturasAsync(codigo);
        return Page();
    }

    // C · Registrar una lectura
    public async Task<IActionResult> OnPostRegistrarAsync(string codigo, decimal valor)
    {
        try
        {
            var (id, situacion) = await repo.RegistrarLecturaAsync(codigo, valor);
            TempData["Ok"] = $"Lectura {id} registrada: {valor:0.00} -> {situacion}.";
        }
        catch (ErrorDatos e) { TempData["Error"] = e.Message; }
        return RedirectToPage(new { codigo });
    }

    // U · Actualizar el umbral
    public async Task<IActionResult> OnPostUmbralAsync(string codigo, decimal umbral)
    {
        try
        {
            var anterior = await repo.ActualizarUmbralAsync(codigo, umbral);
            TempData["Ok"] = $"Umbral de {codigo} actualizado: {anterior:0.00} -> {umbral:0.00}.";
        }
        catch (ErrorDatos e) { TempData["Error"] = e.Message; }
        return RedirectToPage(new { codigo });
    }

    // D · Eliminar una lectura
    public async Task<IActionResult> OnPostEliminarLecturaAsync(string codigo, long id)
    {
        try
        {
            var n = await repo.EliminarLecturaAsync(id);
            TempData["Ok"] = n == 1 ? $"Lectura {id} eliminada." : $"La lectura {id} ya no
existe.";
        }
        catch (ErrorDatos e) { TempData["Error"] = e.Message; }
        return RedirectToPage(new { codigo });
    }
```

```
    // D · Eliminar el sensor (con o sin sus lecturas)
    public async Task<IActionResult> OnPostEliminarSensorAsync(string codigo, bool conLecturas)
    {
        try
        {
            var (lecturas, alertas) = await repo.EliminarSensorAsync(codigo, conLecturas);
            TempData["Ok"] = $"Sensor {codigo} eliminado junto con {lecturas} lecturas y {alertas}
alertas (una transaccion).";
            return RedirectToPage("/Index");
        }
        catch (ErrorDatos e)
        {
            TempData["Error"] = e.Message + (conLecturas ? "" : " Usa \"Eliminar con
lecturas\".");
            return RedirectToPage(new { codigo });
        }
    }
}
```

```
using System.Data;
using IotAdoNet.Datos;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace IotAdoNet.Pages;

/// <summary>Modelo desconectado: DataTable llenado por un DataAdapter y guardado con
Update.</summary>
public class DispositivosModel(RepositorioIot repo) : PageModel
{
    public DataTable Tabla { get; private set; } = new();

    public async Task OnGetAsync() => Tabla = await repo.DispositivosTablaAsync();

    public async Task<IActionResult> OnPostAsync(Dictionary<string, string> estado)
    {
        try
        {
            var (n, sql) = await repo.GuardarEstadosAsync(estado);
            TempData["Ok"] = $"DataAdapter.Update envio {n} fila(s) modificada(s). UPDATE generado
por el CommandBuilder: {sql}";
        }
        catch (ErrorDatos e) { TempData["Error"] = e.Message; }
        return RedirectToPage();
    }
}
```

4.8 Cliente en Visual Basic .NET: Program.vb

```
' ======================================================================
' TAREA 11 · ADO.NET con Visual Basic .NET
' IoT Analytics Dashboard (NexoDev Solutions)
'
' Cliente de consola que usa el MODELO DESCONECTADO de ADO.NET:
'   reporte              DataSet con dos DataTable y un DataRelation
'   ajustar <tipo> <%>   cambia umbrales en memoria y los guarda con DataAdapter.Update
'   xml                  exporta el DataSet a XML (WriteXml)
'   lector               la misma consulta con DataReader (modelo conectado)
' ======================================================================
Imports System.Data
Imports System.Data.Common
Imports Npgsql

Module Program
```

```
    Private Fabrica As DbProviderFactory
    Private Cadena As String

    Function Main(args As String()) As Integer
        ' Mismo modelo de proveedores que el cliente en C#
        DbProviderFactories.RegisterFactory("Npgsql", NpgsqlFactory.Instance)
        Fabrica = DbProviderFactories.GetFactory("Npgsql")
        Cadena = Environment.GetEnvironmentVariable("ConnectionStrings__Iot")
        If String.IsNullOrEmpty(Cadena) Then
            Console.Error.WriteLine("Falta la variable ConnectionStrings__Iot")
            Return 2
        End If

        Dim comando = If(args.Length > 0, args(0), "reporte")
        Try
            Select Case comando
                Case "reporte" : Reporte()
                Case "ajustar"
                    If args.Length < 3 Then
                        Console.WriteLine("Uso: reporte-vb ajustar <tipo> <porcentaje>")
                        Return 1
                    End If
                    Ajustar(args(1), Decimal.Parse(args(2),
Globalization.CultureInfo.InvariantCulture))
                Case "xml" : ExportarXml()
                Case "lector" : Lector()
                Case Else
                    Console.WriteLine("Comandos: reporte | ajustar <tipo> <porcentaje> | xml |
lector")
                    Return 1
            End Select
            Return 0
        Catch ex As DbException
            Console.Error.WriteLine($"Error de base de datos [SQLSTATE {ex.SqlState}]:
{ex.Message}")
            Return 1
        End Try
    End Function

    Private Function Conectar() As DbConnection
        Dim cn = Fabrica.CreateConnection()
        cn.ConnectionString = Cadena
        cn.Open()
        Return cn
    End Function

    Private Function Adaptador(cn As DbConnection, sql As String) As DbDataAdapter
        Dim cmd = cn.CreateCommand()
        cmd.CommandText = sql
        Dim da = Fabrica.CreateDataAdapter()
        da.SelectCommand = cmd
        Return da
    End Function

    ''' <summary>Carga dos tablas en un DataSet, las relaciona y las recorre sin
conexion.</summary>
    Private Function CargarDataSet() As DataSet
        Dim ds As New DataSet("IotAnalytics")
        Using cn = Conectar()
            Adaptador(cn, "SELECT id, codigo, nombre, ubicacion, estado FROM dispositivos ORDER BY
codigo") _
                .Fill(ds, "dispositivos")
            Adaptador(cn, "SELECT s.id, s.dispositivo_id, s.codigo, s.tipo, s.unidad,
s.umbral_max, " &
                          "(SELECT ROUND(AVG(l.valor), 2) FROM lecturas l " &
                          " WHERE l.sensor_id = s.id AND l.medido_en > NOW() - INTERVAL '1 hour')
AS promedio_hora " &
                          "FROM sensores s ORDER BY s.codigo") _
```

```
                .Fill(ds, "sensores")
        End Using                                   ' la conexion se cierra aqui

        ' DataRelation: une las tablas en memoria, como una llave foranea
        ds.Relations.Add("dispositivo_sensores",
                         ds.Tables("dispositivos").Columns("id"),
                         ds.Tables("sensores").Columns("dispositivo_id"))
        Return ds
    End Function

    Private Sub Reporte()
        Dim ds = CargarDataSet()
        Console.WriteLine("REPORTE DE DISPOSITIVOS Y SENSORES (DataSet desconectado)")
        Console.WriteLine(New String("="c, 64))
        For Each d As DataRow In ds.Tables("dispositivos").Rows
            Console.WriteLine($"{d("codigo")} - {d("nombre")} ({d("ubicacion")}) [{d("estado")}]")
            For Each s As DataRow In d.GetChildRows("dispositivo_sensores")
                Dim promedio = If(s.IsNull("promedio_hora"), "sin datos",
                                  CDec(s("promedio_hora")).ToString("0.00"))
                Console.WriteLine($"   {s("codigo"),-11} {s("tipo"),-12} umbral
{CDec(s("umbral_max")),8:0.00} {s("unidad"),-5}" &
                                  $" promedio 1 h: {promedio}")
            Next
        Next
        ' Calculo sobre los datos en memoria, sin volver a consultar la base
        Dim total = ds.Tables("sensores").Compute("COUNT(id)", "")
        Console.WriteLine(New String("-"c, 64))
        Console.WriteLine($"{ds.Tables("dispositivos").Rows.Count} dispositivos, {total}
sensores")
    End Sub

    ''' <summary>
    ''' Ajusta en memoria el umbral de todos los sensores de un tipo y guarda los cambios
    ''' con DataAdapter.Update. El CommandBuilder genera el UPDATE y solo se envian las
    ''' filas con RowState = Modified, todo dentro de una transaccion.
    ''' </summary>
    Private Sub Ajustar(tipo As String, porcentaje As Decimal)
        Using cn = Conectar()
            Using tx = cn.BeginTransaction()
                Dim da = Adaptador(cn, "SELECT id, codigo, tipo, umbral_max FROM sensores ORDER BY
codigo")
                da.SelectCommand.Transaction = tx
                Dim cb = Fabrica.CreateCommandBuilder()
                cb.DataAdapter = da

                Dim tabla As New DataTable("sensores")
                da.Fill(tabla)

                For Each fila As DataRow In tabla.Select($"tipo = '{tipo.Replace("'", "''")}'")
                    Dim antes = CDec(fila("umbral_max"))
                    fila("umbral_max") = Math.Round(antes * (1D + porcentaje / 100D), 2)
                    Console.WriteLine($"  {fila("codigo"),-11} {antes,8:0.00} ->
{CDec(fila("umbral_max")),8:0.00}  RowState = {fila.RowState}")
                Next

                Dim modificadas = tabla.GetChanges(DataRowState.Modified)
                Console.WriteLine($"Filas modificadas en memoria: {If(modificadas Is Nothing, 0,
modificadas.Rows.Count)}")

                Dim upd = cb.GetUpdateCommand()
                upd.Transaction = tx
                da.UpdateCommand = upd
                Dim enviadas = da.Update(tabla)
                tx.Commit()
                Console.WriteLine($"DataAdapter.Update envio {enviadas} UPDATE al servidor y se
confirmo la transaccion.")
                Console.WriteLine($"UPDATE generado por el CommandBuilder:{Environment.NewLine}
{upd.CommandText}")
```

```
            End Using
        End Using
    End Sub

    ''' <summary>ADO.NET y XML: el DataSet se puede guardar o enviar como XML.</summary>
    Private Sub ExportarXml()
        Dim ds = CargarDataSet()
        ds.Relations.Clear()
        ds.Tables("sensores").Columns.Remove("promedio_hora")
        ds.WriteXml(Console.Out, XmlWriteMode.IgnoreSchema)
        Console.WriteLine()
    End Sub

    ''' <summary>Modelo conectado: DataReader recorre el resultado mientras la conexion esta
abierta.</summary>
    Private Sub Lector()
        Using cn = Conectar()
            Using cmd = cn.CreateCommand()
                cmd.CommandText = "SELECT s.codigo, COUNT(l.id), MAX(l.valor) FROM sensores s " &
                                  "LEFT JOIN lecturas l ON l.sensor_id = s.id AND l.medido_en >
NOW() - INTERVAL '10 minutes' " &
                                  "WHERE s.tipo = @tipo GROUP BY s.codigo ORDER BY s.codigo"
                Dim p = cmd.CreateParameter()
                p.ParameterName = "@tipo"
                p.Value = "temperatura"
                cmd.Parameters.Add(p)
                Using dr = cmd.ExecuteReader()
                    Console.WriteLine("Sensores de temperatura, ultimos 10 minutos (DataReader):")
                    While dr.Read()
                        Dim maximo = If(dr.IsDBNull(2), "-", dr.GetDecimal(2).ToString("0.00"))
                        Console.WriteLine($"  {dr.GetString(0),-11} {dr.GetInt64(1),5} lecturas
maximo {maximo}")
                    End While
                End Using
            End Using
        End Using
    End Sub

End Module
```

4.9 Base de datos: init.sql

```
-- ================================================================
-- TAREA 11 · Programacion del lado del cliente: ADO.NET
-- IoT Analytics Dashboard (NexoDev Solutions) · PostgreSQL 16
-- Base de datos que administran los clientes ADO.NET en C# y Visual Basic.
-- ================================================================

CREATE TABLE dispositivos (
    id        SERIAL PRIMARY KEY,
    codigo    VARCHAR(20) NOT NULL UNIQUE,
    nombre    VARCHAR(60) NOT NULL,
    ubicacion VARCHAR(60) NOT NULL,
    estado    VARCHAR(15) NOT NULL DEFAULT 'activo'
              CHECK (estado IN ('activo', 'mantenimiento', 'inactivo'))
);

CREATE TABLE sensores (
    id             SERIAL PRIMARY KEY,
    dispositivo_id INT NOT NULL REFERENCES dispositivos(id),
    codigo         VARCHAR(20) NOT NULL UNIQUE,
    tipo           VARCHAR(20) NOT NULL CHECK (tipo IN ('temperatura', 'vibracion', 'energia',
'humedad')),
    unidad         VARCHAR(10) NOT NULL,
    umbral_max     NUMERIC(10,2) NOT NULL CHECK (umbral_max > 0)
);
```

```
CREATE TABLE lecturas (
    id        BIGSERIAL PRIMARY KEY,
    sensor_id INT NOT NULL REFERENCES sensores(id),
    valor     NUMERIC(10,2) NOT NULL,
    medido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);
CREATE INDEX idx_lecturas_fecha        ON lecturas (medido_en DESC);

CREATE TABLE alertas (
    id         SERIAL PRIMARY KEY,
    lectura_id BIGINT NOT NULL REFERENCES lecturas(id) ON DELETE CASCADE,
    sensor_id  INT NOT NULL REFERENCES sensores(id),
    severidad  VARCHAR(12) NOT NULL CHECK (severidad IN ('advertencia', 'critica')),
    valor      NUMERIC(10,2) NOT NULL,
    umbral     NUMERIC(10,2) NOT NULL,
    creada_en  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atendida   BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_alertas_lectura ON alertas (lectura_id);
CREATE INDEX idx_alertas_pendientes ON alertas (creada_en DESC) WHERE NOT atendida;

-- Bitacora de los cambios hechos desde los clientes ADO.NET (se muestra en Grafana)
CREATE TABLE auditoria (
    id          BIGSERIAL PRIMARY KEY,
    tabla       TEXT NOT NULL,
    operacion   TEXT NOT NULL,
    registro    TEXT,
    cambios     JSONB,
    aplicacion  TEXT NOT NULL DEFAULT current_setting('application_name'),
    ocurrido_en TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

-- Alerta automatica al insertar una lectura sobre el umbral
CREATE FUNCTION trg_lecturas_alerta() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_umbral NUMERIC;
BEGIN
    SELECT umbral_max INTO v_umbral FROM sensores WHERE id = NEW.sensor_id;
    IF NEW.valor > v_umbral THEN
        INSERT INTO alertas (lectura_id, sensor_id, severidad, valor, umbral, creada_en)
        VALUES (NEW.id, NEW.sensor_id,
                CASE WHEN NEW.valor > v_umbral * 1.2 THEN 'critica' ELSE 'advertencia' END,
                NEW.valor, v_umbral, NEW.medido_en);
    END IF;
    RETURN NULL;
END $$;

CREATE TRIGGER t_lecturas_alerta AFTER INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION trg_lecturas_alerta();

-- Auditoria de altas, cambios y bajas de sensores y dispositivos.
-- application_name identifica que cliente hizo el cambio (web C# o reporte VB).
CREATE FUNCTION trg_auditar() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_cambios JSONB;
BEGIN
    IF TG_OP = 'UPDATE' THEN
        SELECT jsonb_object_agg(n.key, jsonb_build_object('antes', o.value, 'despues', n.value))
          INTO v_cambios
          FROM jsonb_each(to_jsonb(NEW)) n JOIN jsonb_each(to_jsonb(OLD)) o USING (key)
         WHERE n.value IS DISTINCT FROM o.value;
        IF v_cambios IS NULL THEN RETURN NULL; END IF;
    ELSIF TG_OP = 'INSERT' THEN
        v_cambios := to_jsonb(NEW);
    ELSE
```

```
        v_cambios := to_jsonb(OLD);
    END IF;
    INSERT INTO auditoria (tabla, operacion, registro, cambios)
    VALUES (TG_TABLE_NAME, TG_OP, COALESCE(to_jsonb(NEW), to_jsonb(OLD)) ->> 'codigo', v_cambios);
    RETURN NULL;
END $$;

CREATE TRIGGER t_auditar_sensores AFTER INSERT OR UPDATE OR DELETE ON sensores
FOR EACH ROW EXECUTE FUNCTION trg_auditar();
CREATE TRIGGER t_auditar_dispositivos AFTER INSERT OR UPDATE OR DELETE ON dispositivos
FOR EACH ROW EXECUTE FUNCTION trg_auditar();

-- Usuario de los clientes ADO.NET: CRUD sobre los datos, sin cambiar la estructura
CREATE ROLE app_adonet LOGIN PASSWORD 'adonet123';
GRANT SELECT, INSERT, UPDATE, DELETE ON dispositivos, sensores, lecturas, alertas TO app_adonet;
GRANT SELECT ON auditoria TO app_adonet;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_adonet;

-- Grafana: solo lectura
CREATE ROLE grafana_lector LOGIN PASSWORD 'grafana123';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO grafana_lector;
-- Las tablas de Grafana muestran horas como texto (to_char): se calculan en la zona local
ALTER ROLE grafana_lector SET timezone = 'America/Mexico_City';

-- Datos iniciales (la carga inicial no se audita)
ALTER TABLE dispositivos DISABLE TRIGGER t_auditar_dispositivos;
ALTER TABLE sensores DISABLE TRIGGER t_auditar_sensores;

INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidraulica 3', 'Linea 1'),
('TOR-01', 'Torno CNC 1',         'Linea 2'),
('EMB-02', 'Embotelladora 2',     'Linea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);

ALTER TABLE dispositivos ENABLE TRIGGER t_auditar_dispositivos;
ALTER TABLE sensores ENABLE TRIGGER t_auditar_sensores;

-- 6 horas de historico (una lectura cada 30 s por sensor)
SELECT setseed(0.6);
INSERT INTO lecturas (sensor_id, valor, medido_en)
SELECT s.id, ROUND((s.umbral_max * (0.80 + (random() - 0.5) * 0.08))::numeric, 2), t
FROM sensores s
CROSS JOIN generate_series(NOW() - INTERVAL '6 hours', NOW() - INTERVAL '1 minute',
                           INTERVAL '30 seconds') AS t;
ANALYZE;
```

4.10 Docker

```
# Tarea 11 · ADO.NET con C# y Visual Basic · IoT Analytics Dashboard (NexoDev Solutions)
name: iot-adonet

services:
  # ================ BASE DE DATOS ================
  datos:
    image: postgres:16
    container_name: adonet-datos
    environment:
```

```
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
      POSTGRES_DB: iotanalytics
    command: >
      -c ssl=on -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    volumes:
      - ./datos/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks: [red-datos]          # sin puerto expuesto a la PC
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 30

  # ========= CLIENTE ADO.NET EN C#: interfaz grafica web + simulador =========
  web:
    build: ./web
    container_name: adonet-web
    environment:
      ConnectionStrings__Iot: >-
        Host=datos;Database=iotanalytics;Username=app_adonet;Password=adonet123;
        SSL Mode=Require;Application Name=web-csharp;Timezone=America/Mexico_City
      SIMULADOR: "true"
    ports:
      - "8091:8080"                # http://localhost:8091
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

  # ========= CLIENTE ADO.NET EN VISUAL BASIC: modelo desconectado =========
  vb:
    build: ./vb
    container_name: adonet-vb
    environment:
      ConnectionStrings__Iot: >-
        Host=datos;Database=iotanalytics;Username=app_adonet;Password=adonet123;
        SSL Mode=Require;Application Name=reporte-vb;Timezone=America/Mexico_City
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

  # ================= VISUALIZACION =================
  grafana:
    image: grafana/grafana:11.3.0
    container_name: adonet-grafana
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin123
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/iot-adonet.json
    ports:
      - "3011:3000"                # http://localhost:3011
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

networks:
  red-datos:
```

```
# ---------- Etapa 1: restaurar Npgsql y compilar con el SDK de .NET ----------
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS compilacion
WORKDIR /src
COPY IotAdoNet.csproj .
RUN dotnet restore
COPY . .
```

```
RUN dotnet publish -c Release -o /app --no-restore

# ---------- Etapa 2: solo el runtime de ASP.NET Core ----------
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=compilacion /app .
EXPOSE 8080
ENTRYPOINT ["dotnet", "IotAdoNet.dll"]
```

5. Pruebas y resultados

La base inicia con 6 horas de histórico (6,480 lecturas) y el simulador agrega 9 lecturas por segundo con picos que generan alertas. Cada prueba se hizo desde la interfaz web o desde el cliente de Visual Basic.

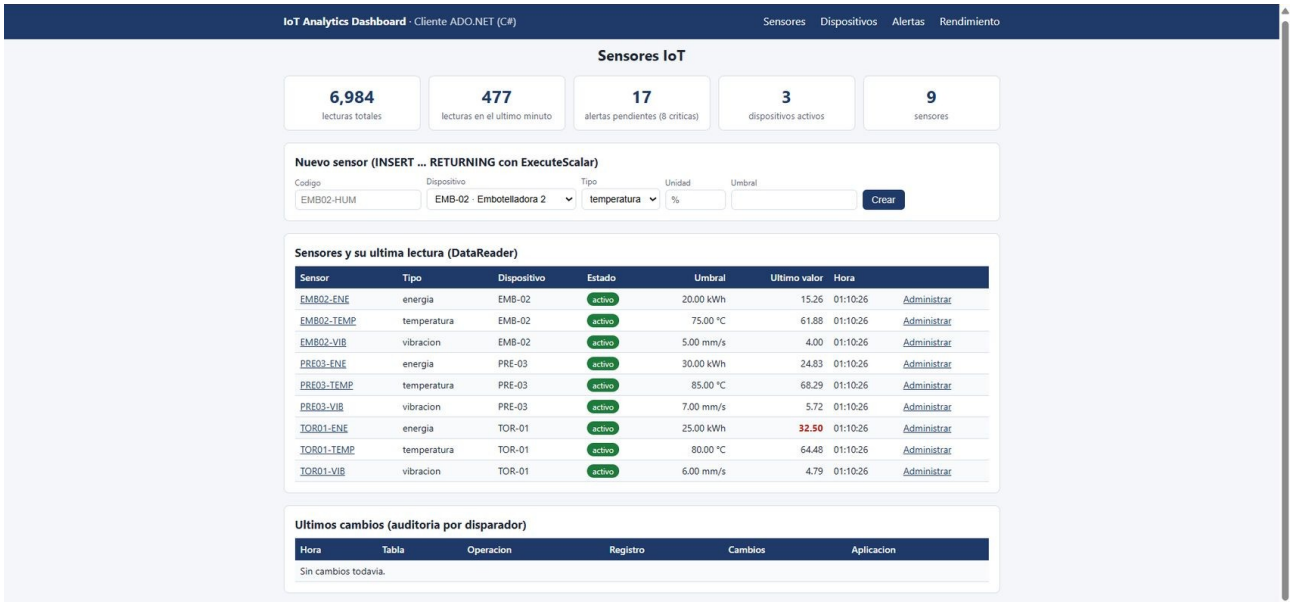
NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Compilación y arranque	Npgsql restaurado; 4 contenedores	Correcto: web y vb compiladas en Docker; base healthy
P2	Conexión y lectura (DataReader)	Resumen y sensores con su última lectura	Correcto: 6,984 lecturas, 477 en el último minuto, 9 sensores
P3	Crear sensor (ExecuteScalar)	Id generado y auditoría	Correcto: EMB02-HUM con id 13, tipo humedad, unidad %
P4	Crear lectura (transacción)	Lectura y alerta del disparador	Correcto: 90 % clasificada como crítica
P5	Actualizar umbral	Valor anterior y nuevo	Correcto: 70.00 → 75.00 y registrado en la auditoría
P6	Eliminar lectura	Fila eliminada	Correcto: lectura 17103 y su alerta (en cascada)
P7	Eliminar sensor con lecturas	Rechazo por llave foránea	Correcto: SQLSTATE 23503, sin cambios
P8	Eliminar sensor y lecturas	Todo en una transacción	Correcto: sensor, 142 lecturas y 2 alertas
P9	Llave duplicada / CHECK	Mensaje claro, sin cambios	Correcto: 23505 y 23514
P10	Modelo desconectado (C#)	Update solo de las filas modificadas	Correcto: 1 fila; UPDATE con concurrencia optimista
P11	Atender alertas (ExecuteNonQuery)	Filas afectadas	Correcto: 383 alertas atendidas
P12	Rendimiento	Agrupar operaciones reduce el tiempo	Correcto: DbBatch 38.5 veces más rápido
P13	DataSet y DataRelation (VB)	Reporte agrupado sin volver a consultar	Correcto: 3 dispositivos y 10 sensores
P14	DataAdapter.Update (VB)	3 filas Modified y 3 UPDATE	Correcto: umbrales de vibración +10 % en una transacción
P15	Grafana	Paneles con los datos y la auditoría	Correcto: gráficas por minuto, alertas y operaciones de web-csharp y reporte-vb

Evidencia 1 — P1: Docker restaura Npgsql, compila los dos clientes y levanta los cuatro contenedores.



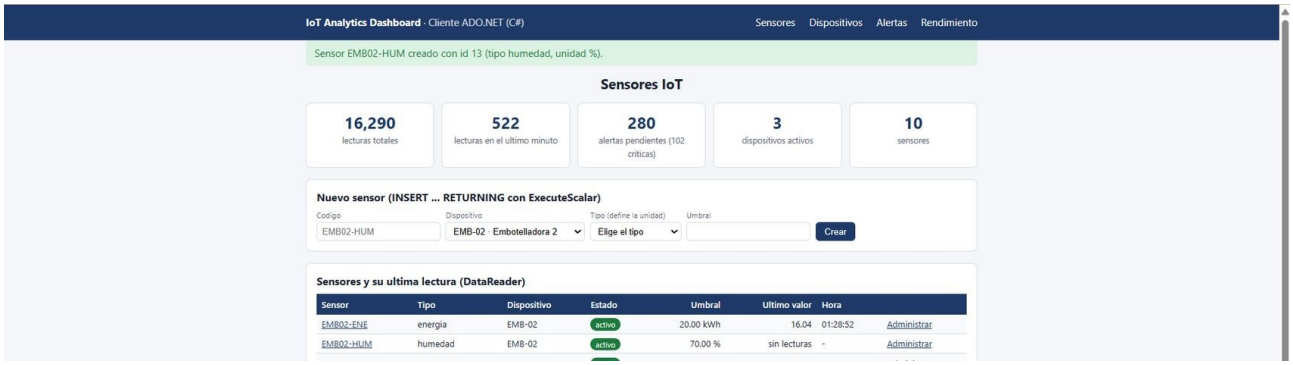
Arranque del entorno.

Evidencia 2 — P2: la interfaz gráfica, conectada con ADO.NET, muestra los indicadores y la última lectura de cada sensor leída con un DbDataReader; los valores nulos de un sensor sin lecturas se detectan con IsDBNull.



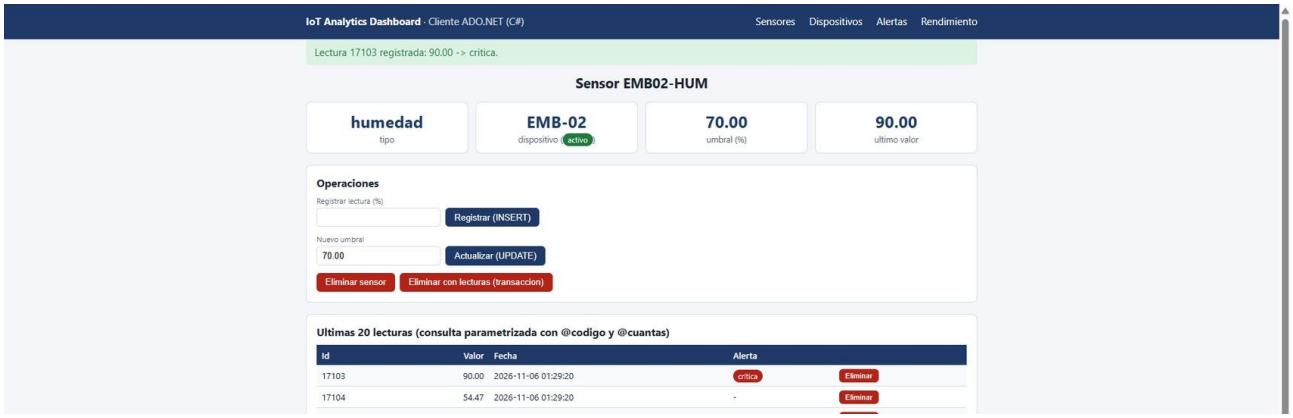
Pantalla principal del cliente ADO.NET.

Evidencia 3 — P3 (Crear): el sensor se inserta con INSERT ... RETURNING y ExecuteScalar devuelve el id generado; la unidad se asignó a partir del tipo.



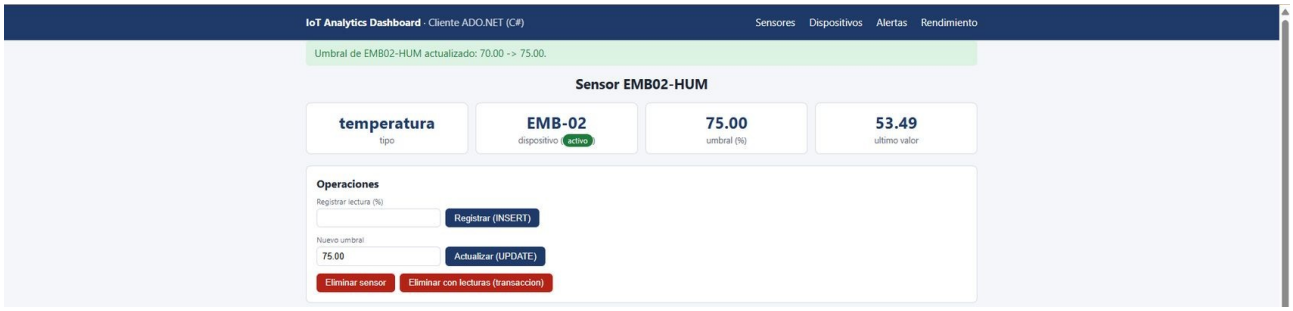
Creación de un sensor.

Evidencia 4 — P4 (Crear): la lectura de 90 % se registra en una transacción y el disparador genera la alerta crítica, que el cliente consulta antes de confirmar.



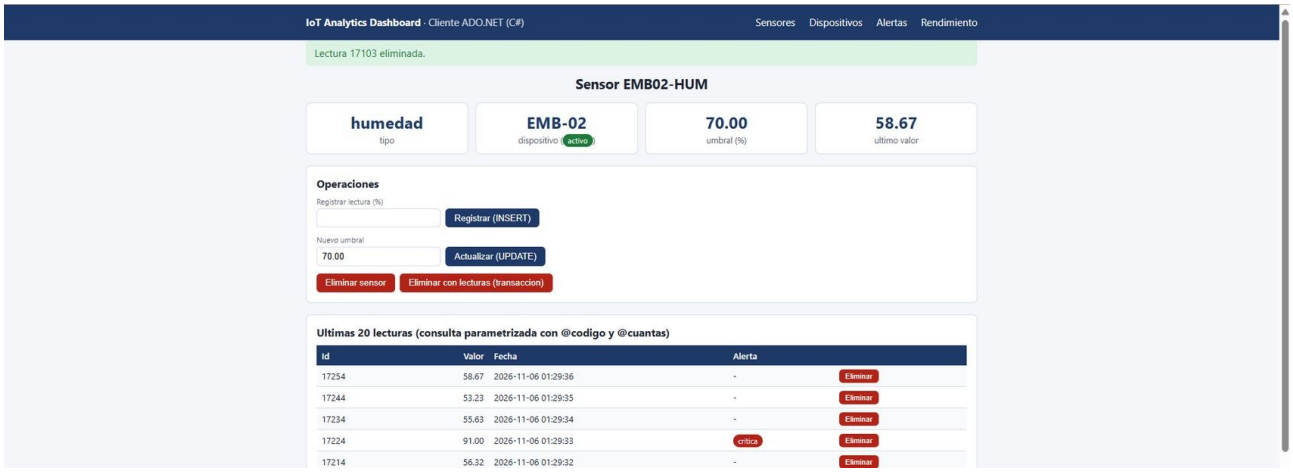
Registro de una lectura con alerta.

Evidencia 5 — P5 (Actualizar): el umbral cambia de 70.00 a 75.00. El tipo “temperatura” que aparece en esta captura es el problema descrito en la sección 7.1, corregido después.



Actualización del umbral.

Evidencia 6 — P6 (Eliminar): la lectura 17103 se elimina y su alerta desaparece por el ON DELETE CASCADE.



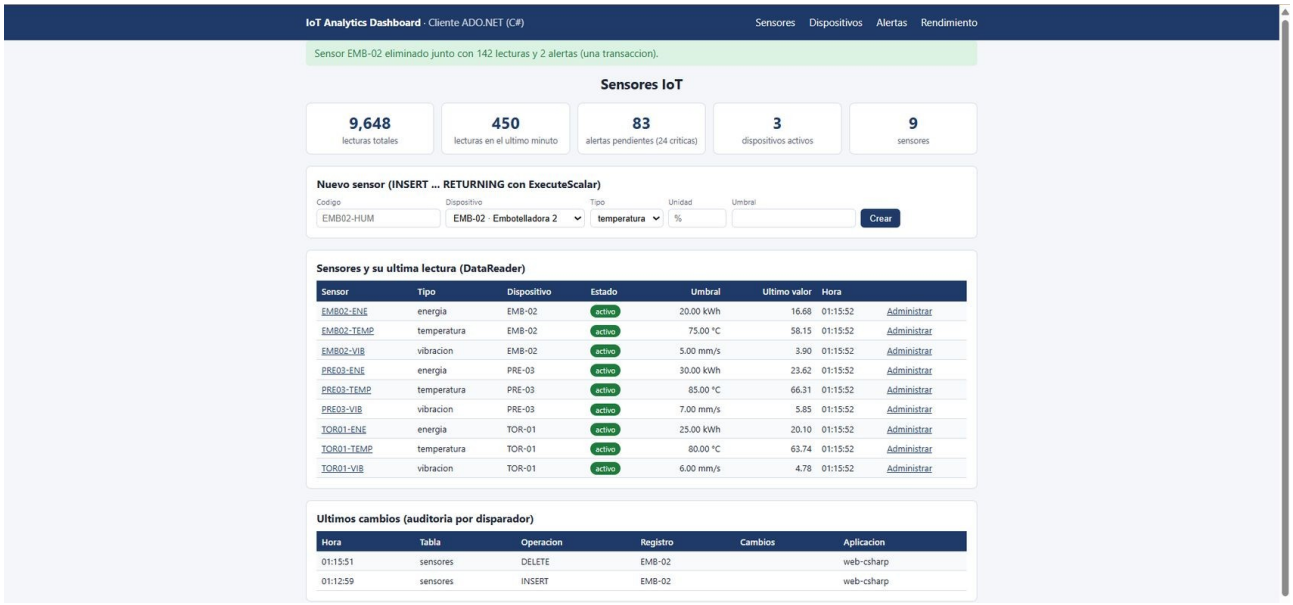
Eliminación de una lectura.

Evidencia 7 — P7: eliminar un sensor que tiene lecturas viola la llave foránea; el error se traduce a un mensaje claro con su SQLSTATE y no se borra nada.



Rechazo por llave foránea.

Evidencia 8 — P8 (Eliminar): con la opción “con lecturas”, alertas, lecturas y sensor se eliminan en una sola transacción. La auditoría registra el DELETE hecho por web-csharp.



Eliminación en una transacción.

Evidencia 9 — P9: crear un sensor con un código existente viola la restricción UNIQUE (23505).



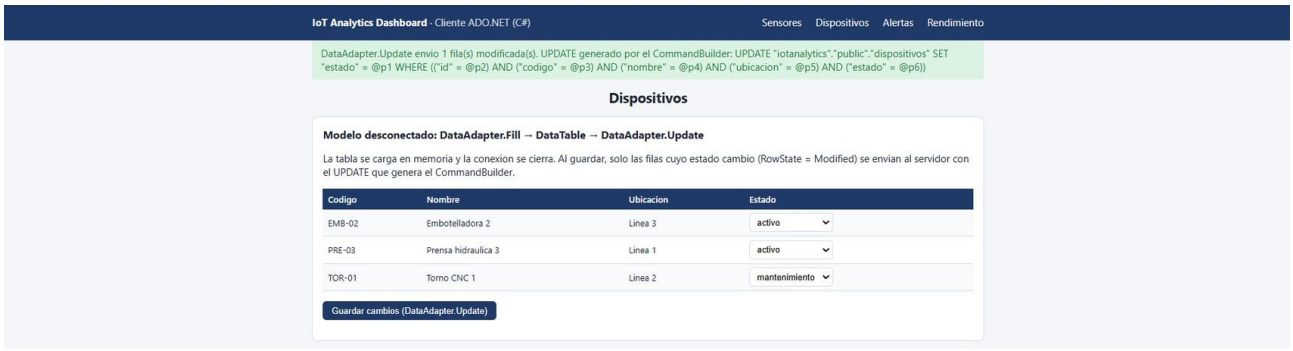
Llave duplicada.

Evidencia 10 — P9: un umbral negativo viola la restricción CHECK (23514).



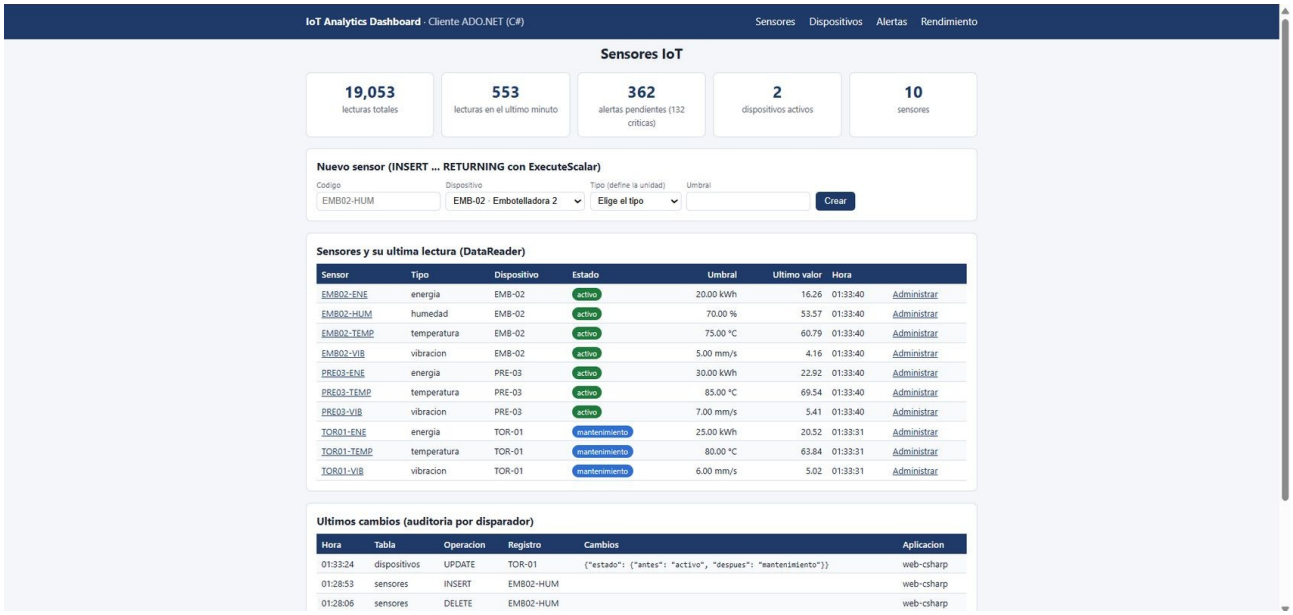
Restricción CHECK.

Evidencia 11 — P10 (modelo desconectado): los dispositivos se cargan en un DataTable con DataAdapter.Fill, se cambia TOR-01 en memoria y DataAdapter.Update envía solo esa fila. El UPDATE que generó el CommandBuilder compara en el WHERE todos los valores originales: así implementa la concurrencia optimista.



DataAdapter.Update con CommandBuilder.

Evidencia 12 — P10: TOR-01 pasa a mantenimiento, el simulador deja de enviar lecturas de sus sensores (su hora se detiene en 01:33:31) y la auditoría registra el cambio con los valores de antes y después.



Efecto del cambio de estado.

Evidencia 13 — P11: ExecuteNonQuery devuelve el número de filas afectadas por el UPDATE.



Atención de alertas.

Evidencia 14 — P12: la prueba de rendimiento inserta 1,000 lecturas de cuatro formas (sección 6).



Prueba de rendimiento.

Evidencia 15 — P13 y P14 (Visual Basic .NET): el reporte recorre un DataSet con dos DataTable unidas por una DataRelation, sin volver a consultar la base; el comando ajustar modifica los umbrales de vibración en memoria (RowState = Modified) y DataAdapter.Update envía 3 UPDATE en una transacción; lector usa el modelo conectado con un DataReader.

```
C:\Users\ASLHY\Downloads\iot-adonet\iot-adonet>docker exec adonet-vb reporte-vb reporte
REPORTE DE DISPOSITIVOS Y SENSORES (DataSet desconectado)
============================================================
EMB-02 - Embotelladora 2 (Linea 3) [activo]
   EMB02-ENE   energia      umbral    20.00 kWh   promedio 1 h: 16.17
   EMB02-HUM   humedad      umbral    70.00 %     promedio 1 h: 56.66
   EMB02-TEMP  temperatura  umbral    75.00 °C    promedio 1 h: 60.93
   EMB02-VIB   vibracion    umbral     5.00 mm/s  promedio 1 h: 4.05
PRE-03 - Prensa hidraulica 3 (Linea 1) [activo]
   PRE03-ENE   energia      umbral    30.00 kWh   promedio 1 h: 24.29
   PRE03-TEMP  temperatura  umbral    85.00 °C    promedio 1 h: 69.00
   PRE03-VIB   vibracion    umbral     7.00 mm/s  promedio 1 h: 5.68
TOR-01 - Torno CNC 1 (Linea 2) [activo]
   TOR01-ENE   energia      umbral    25.00 kWh   promedio 1 h: 20.25
   TOR01-TEMP  temperatura  umbral    80.00 °C    promedio 1 h: 64.70
   TOR01-VIB   vibracion    umbral     6.00 mm/s  promedio 1 h: 4.86
------------------------------------------------------------
3 dispositivos, 10 sensores

C:\Users\ASLHY\Downloads\iot-adonet\iot-adonet>docker exec adonet-vb reporte-vb ajustar vibracion 10
  EMB02-VIB       5.00 ->     5.50  RowState = Modified
  PRE03-VIB       7.00 ->     7.70  RowState = Modified
  TOR01-VIB       6.00 ->     6.60  RowState = Modified
Filas modificadas en memoria: 3
DataAdapter.Update envio 3 UPDATE al servidor y se confirmo la transaccion.
UPDATE generado por el CommandBuilder:
  UPDATE "iotanalytics"."public"."sensores" SET "umbral_max" = @p1 WHERE (("id" = @p2) AND ("codigo" = @p3) AND ("tipo" = @p4) AND ("umbral_max" = @p5))

C:\Users\ASLHY\Downloads\iot-adonet\iot-adonet>docker exec adonet-vb reporte-vb lector
Sensores de temperatura, ultimos 10 minutos (DataReader):
  EMB02-TEMP    577 lecturas   maximo 97.50
  PRE03-TEMP    577 lecturas   maximo 110.50
  TOR01-TEMP    517 lecturas   maximo 104.00

C:\Users\ASLHY\Downloads\iot-adonet\iot-adonet>docker exec adonet-vb reporte-vb xml
```

Cliente ADO.NET en Visual Basic.

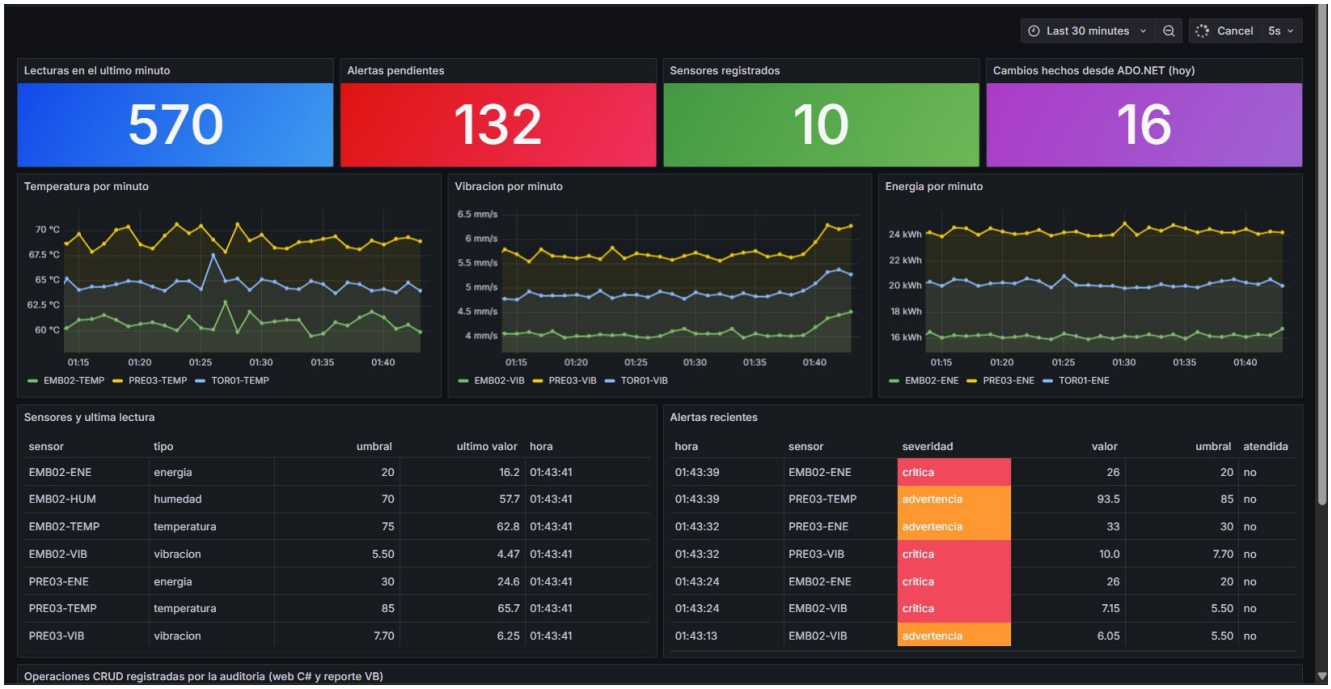
Evidencia 16: el DataSet exportado a XML con WriteXml, con los nuevos umbrales de vibración (7.70 y 6.60).

```
  <sensores>
    <id>4</id>
    <dispositivo_id>2</dispositivo_id>
    <codigo>TOR01-TEMP</codigo>
    <tipo>temperatura</tipo>
    <unidad>°C</unidad>
    <umbral_max>80.00</umbral_max>
  </sensores>
  <sensores>
    <id>5</id>
    <dispositivo_id>2</dispositivo_id>
    <codigo>TOR01-VIB</codigo>
    <tipo>vibracion</tipo>
    <unidad>mm/s</unidad>
    <umbral_max>6.60</umbral_max>
  </sensores>
</IotAnalytics>

C:\Users\ASLHY\Downloads\iot-adonet\iot-adonet>
```

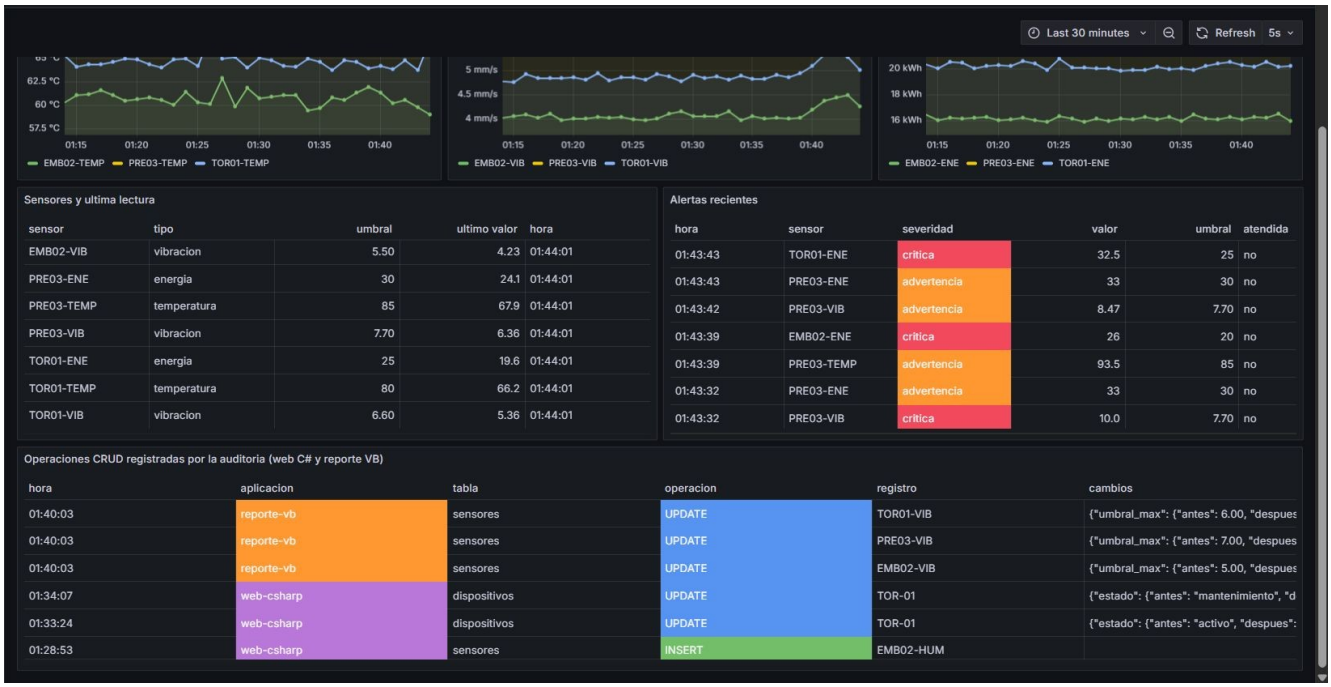
Exportación del DataSet a XML.

Evidencia 17 — P15: Grafana muestra las lecturas del último minuto, las alertas pendientes y las gráficas por tipo de sensor a partir de los datos que guardan los clientes ADO.NET.



Tablero de monitoreo en Grafana.

Evidencia 18 — P15: el panel de auditoría distingue las operaciones CRUD de cada cliente por su Application Name: reporte-vb (naranja) y web-csharp (morado).



Auditoría de las operaciones de los clientes.

6. Análisis de rendimiento

La página Rendimiento inserta 1,000 lecturas de cuatro formas con ADO.NET y luego las elimina:

FORMA	TOTAL	POR LECTURA	MEJORA
A. Un comando por lectura (autocommit)	8,118.4 ms	8.118 ms	—
B. Un comando por lectura, una transacción	753.9 ms	0.754 ms	10.8 veces
C. Comando preparado, una transacción	598.3 ms	0.598 ms	13.6 veces

FORMA	TOTAL	POR LECTURA	MEJORA
D. DbBatch (un solo viaje al servidor)	211.0 ms	0.211 ms	38.5 veces

Interpretación: sin transacción explícita cada INSERT es su propia transacción, y cada COMMIT obliga a PostgreSQL a escribir en disco; con Docker Desktop en Windows esa escritura es el costo dominante, igual que se midió en la Tarea 10 con C. Agruparlas en una `DbTransaction` (B) reduce el tiempo 10.8 veces. `Prepare` (C) ahorra además el análisis del SQL en cada ejecución. `DbBatch` (D) envía las 1,000 sentencias en un solo viaje de red, eliminando la espera de ida y vuelta por cada una.

Decisión aplicada: el simulador usa un comando preparado y una transacción por ronda, y el cliente de Visual Basic guarda todos los cambios de `DataAdapter.Update` en una sola transacción.

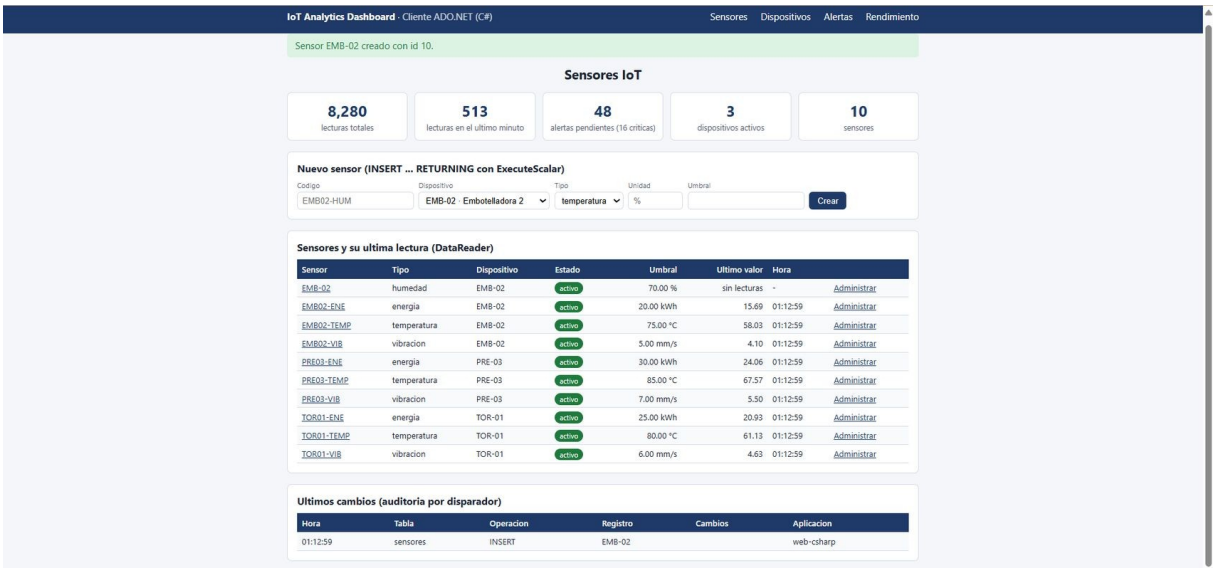
6.1 Impacto en el IoT Analytics Dashboard

ASPECTO	APORTE DEL MÓDULO ADO.NET
Gestión de datos	CRUD completo sobre sensores, lecturas, alertas y dispositivos desde una interfaz gráfica
Tiempo real	El simulador registra 9 lecturas por segundo y la interfaz y Grafana las muestran al instante
Integridad	Transacciones en cada operación de varios pasos; errores traducidos y deshechos
Seguridad	SSL, consultas parametrizadas y un usuario sin permisos sobre la estructura
Trazabilidad	La auditoría registra quién (qué aplicación) hizo cada cambio y qué cambió
Eficiencia	Agrupar escrituras reduce el tiempo hasta 38.5 veces

7. Problemas encontrados y solución

7.1 Se podía guardar un sensor con tipo y unidad incompatibles

Al probar la creación de sensores, el formulario permitía elegir cualquier tipo y escribir cualquier unidad. Así quedó guardado EMB02-HUM con tipo “temperatura” y unidad “%”, porque el selector de tipo se quedó en su valor por defecto. La base de datos lo aceptó porque cada columna, por separado, era válida. Antes también se había creado un sensor con el código del dispositivo (EMB-02) en lugar del suyo.



Sensor creado por error con el código del dispositivo.

Solución: la unidad ya no se captura a mano: el repositorio la deriva del tipo con un diccionario (`UnidadPorTipo`), y el selector de tipo empieza vacío y es obligatorio, mostrando la unidad que corresponde. Los sensores incorrectos se eliminaron con la opción “con lecturas”, lo que también sirvió como prueba de eliminación (Evidencia 8). La Evidencia 3 muestra el sensor ya creado correctamente.



Recompilación solo del contenedor web tras la corrección.

7.2 El conteo de alertas pendientes era incorrecto

La página de alertas mostraba “50 alertas pendientes”, pero al atenderlas `ExecuteNonQuery` informó 383 filas actualizadas. El número salía del tamaño de la lista, limitada a 50 filas con `LIMIT`, no del total real.



Antes: el conteo se limitaba a 50.

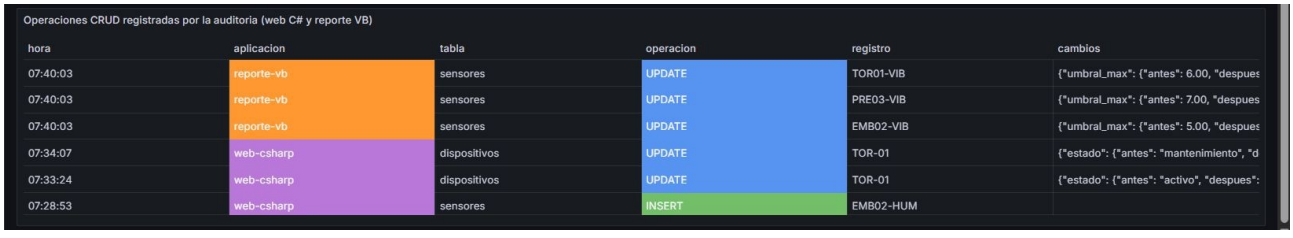
Solución: el total se obtiene con un `COUNT(*)` aparte y la página indica cuántas muestra. Este error ilustra la diferencia entre el tamaño de un resultado leído con un DataReader y el número real de registros.



Después: 96 alertas pendientes, mostrando 50.

7.3 Horas desfasadas en Grafana

Las tablas de Grafana mostraban las 07:41 cuando en México eran las 01:41. El usuario `grafana_lector` trabajaba en UTC y, como las horas se convierten a texto con `to_char` dentro de la consulta, Grafana ya no podía ajustarlas a la zona horaria del navegador.



Antes: horas en UTC en el panel de auditoría.

Solución: `ALTER ROLE grafana_lector SET timezone = 'America/Mexico_City'`, agregado también a `init.sql`. Se aplicó sin reconstruir la base, reiniciando solo Grafana (Evidencias 17 y 18).

7.4 Sin acceso a NuGet durante el desarrollo

El entorno donde se escribió el código no tenía acceso al repositorio de paquetes NuGet, así que no podía descargar Npgsql. Para no entregar código sin verificar, se compilaron ambos proyectos contra un sustituto mínimo de Npgsql (para comprobar la sintaxis y los tipos de ADO.NET) y se ejecutaron las 24 consultas SQL del repositorio contra PostgreSQL con los permisos reales del usuario `app_adonet`. La compilación final con Npgsql la hizo Docker y funcionó a la primera (Evidencia 1).

8. Control de versiones y colaboración

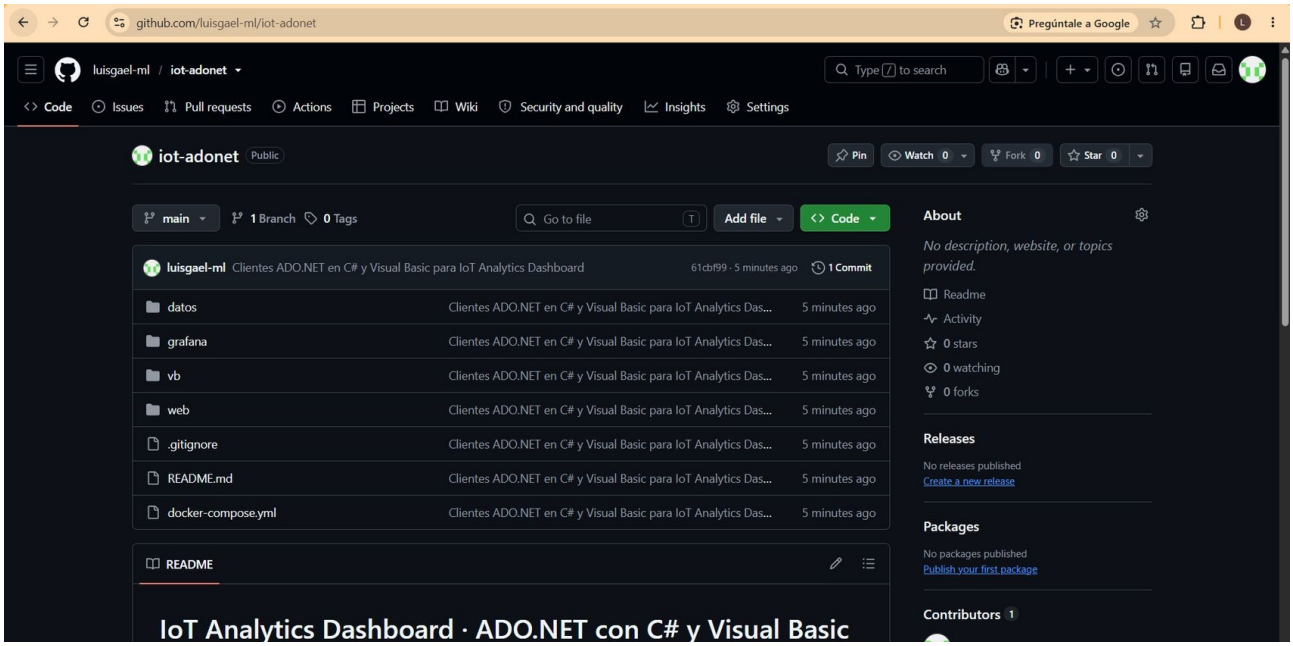
La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público con un README que explica los dos clientes, los modelos de ADO.NET y cómo levantar el entorno: https://github.com/luisgael-ml/iot-adonet

```
git init
git config user.name "Luis Gael Monterrubio López"
git config user.email "monterrubiolopezluisgael@gmail.com"
git add .
git commit -m "Clientes ADO.NET en C# y Visual Basic para IoT Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-adonet.git
git push -u origin main
```

Evidencia 19: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.



Publicación del proyecto en GitHub.



Repositorio iot-adonet.

8.1 Reflexión individual sobre la colaboración

Busqué soluciones en lugar de detenerme ante los obstáculos: cuando no pude descargar Npgsql, encontré la forma de comprobar el código igualmente; y cada vez que una prueba mostró algo raro —una unidad que no correspondía, un conteo que no cuadraba, horas desfasadas— lo traté como un error a corregir y no como un detalle, porque un compañero que use este módulo confiaría en esos datos.

Pensando en ayudar a otros a implementar ADO.NET, preparé el proyecto como material de referencia: cada pantalla indica en su título qué clase o técnica usa (DataReader, ExecuteScalar, DataAdapter.Update), la página de dispositivos muestra el SQL que genera el CommandBuilder, la de rendimiento permite comparar las cuatro formas de insertar, y el cliente de Visual Basic demuestra que las mismas clases funcionan igual en otro lenguaje de .NET. También incluí en el marco teórico el mismo ejemplo en Visual Basic, C# y C++, para que quien venga de cualquiera de esos lenguajes encuentre su punto de partida, y documenté los cuatro problemas con su causa y su solución.

9. Conclusiones

ADO y ADO.NET resuelven el mismo problema —conectar un programa con una base de datos— con filosofías distintas. ADO clásico, basado en COM, gira alrededor de un Recordset conectado y vive en aplicaciones heredadas de Windows en Visual Basic 6 y C++. ADO.NET separa los proveedores de datos del DataSet en memoria, trabaja desconectado por diseño, usa XML y, con .NET 8, corre en cualquier sistema, como se comprobó al ejecutarlo en contenedores Linux.

El módulo desarrollado implementa el CRUD completo sobre los datos IoT desde una interfaz gráfica en C#, usando los dos modelos de ADO.NET: el conectado, con DbCommand y DbDataReader, para las operaciones rápidas y en tiempo real; y el desconectado, con DataAdapter, DataTable y CommandBuilder, para editar datos en memoria y guardarlos con concurrencia optimista. El cliente en Visual Basic .NET confirmó que las clases son las mismas en ambos lenguajes. Las consultas

parametrizadas, las transacciones y la traducción de errores por SQLSTATE mantuvieron la integridad en todas las pruebas, la auditoría mostrada en Grafana dejó trazabilidad de cada cambio, y el análisis de rendimiento mostró que agrupar escrituras con DbBatch fue 38.5 veces más rápido. Así, el módulo cumple el objetivo de NexoDev de gestionar los datos IoT en tiempo real de forma eficiente y confiable.

Referencias

Docker Inc. (s.f.). *Multi-stage builds*. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Grafana Labs. (s.f.). *PostgreSQL data source*. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/postgres/

Microsoft. (s.f.). *ADO.NET overview*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview

Microsoft. (s.f.). *ADO.NET DataSets*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-datasets

Microsoft. (s.f.). *ADO programmer's guide*. Microsoft Learn. https://learn.microsoft.com/en-us/sql/ado/guide/ado-programmer-s-guide

Microsoft. (s.f.). *Comparison of ADO.NET and ADO*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/visualstudio/visual-studio-2008/904fck4k(v=vs.90)

Microsoft. (s.f.). *Generating commands with CommandBuilders*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/generating-commands-with-commandbuilders

Microsoft. (s.f.). *Visual C++ ADO programming*. Microsoft Learn. https://learn.microsoft.com/en-us/office/client-developer/access/desktop-database-reference/visual-c-ado-programming

Npgsql. (s.f.). *Basic usage*. Npgsql documentation. https://www.npgsql.org/doc/basic-usage.html

Npgsql. (s.f.). *Prepared statements*. Npgsql documentation. https://www.npgsql.org/doc/prepare.html

PostgreSQL Global Development Group. (s.f.). *PostgreSQL error codes*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/errcodes-appendix.html